RKNN3-Toolkit API 参考手册

文件标识：RK-YH-YF-412

发布版本：V1.2.0

日期：2026-10-23

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文是 RKNN3-Toolkit 的 API 参考手册。

RKNN3-Toolkit 是为用户提供在PC平台上进行模型转换、推理和性能评估的开发套件。

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明	核定人
V0.2.0	HPC团队	2025-8-6	初始版本	熊伟
V0.3.0b0	NN团队	2025-9-12	更新load_rknn/init_runtime/inference/list_devices接口说明	熊伟
V0.4.0b0	NN团队	2025-10-18	rknn.config 新增 rknn_build_parallelism llm_head_dtype和llm_head_quantized_method参数说明	熊伟
V0.5.0	NN团队	2025-12-01	- 支持连板精度分析，更新init_runtime/inference接口说明. - 新增kvcache_controller/query接口说明	熊伟
V1.0.0	NN团队	2026-01-10	正式版本	熊伟
V1.0.1	NN团队	2026-02-13	增加LoRA支持	熊伟
V1.0.2	NN团队	2026-03-13	增加RK3572平台说明	熊伟
V1.0.4	NN团队	2026-05-11	精度分析新增analysis_core_mask参数、更新llm_config配置项	熊伟
V1.1.0	NN团队	2026-08-05	- 移除Caffe/TensorFlow/TensorFlow Lite/DarkNet/PyTorch框架支持，仅保留ONNX. - 更新系统依赖说明. - 新增reg_custom_op接口说明. - 不再维护Accuracy_analysis接口，请使用inference(accuracy_analysis=True)替代	熊伟
V1.2.0	NN团队	2026-10-23	更新分页 KV Cache、LoRA、MoE、ONNX→PyTorch 模型生成与重建等接口说明	熊伟

目录

RKNN3-Toolkit API 参考手册

1 环境说明

1.1 支持芯片

1.2 系统依赖说明

1.3 适用的深度学习框架

2 RKNN3-Toolkit 调度流程说明

2.1 主链路：模型转换、导出与可选调试

2.2 rebuild 链路：基于构建缓存重新构建模型

2.3 load_ctx 链路：加载上下文并调试已导出模型

2.4 RKNN 对象与资源生命周期

3 模型配置接口

3.1 实验参数

3.2 已弃置参数

3.3 llm_config 配置项

4 模型加载接口

4.1 ONNX模型加载

4.2 LLM模型加载接口

5 模型构建和量化接口

5.1 构建RKNN模型

5.2 重建RKNN模型

6 RKNN模型导出接口

7 运行时初始化与推理接口

7.1 加载RKNN模型

7.2 初始化运行时环境

7.2.1 分配和扩容分页 KV Cache

7.3 注册自定义算子

7.4 模型推理

8 精度、性能和内存评估接口

8.1 精度分析

8.2 评估模型性能

8.3 获取内存使用情况

8.4 获取设备列表

9 LLM和VLM转换接口

9.1 LoRA加载、开启和关闭
9.1.1 LoRA权重加载
9.1.2 开启LoRA
9.1.3 关闭LoRA
9.2 查询功能
9.2.1 查询命令的类型
9.3 KVCache控制器
9.3.1 获取控制kvcache的输入
9.3.2 清空kvcache状态
9.3.3 回退缓存位置
9.3.4 绑定并自动补齐控制输入
10 ONNX→PyTorch 模型生成与重建
10.1 生成 PyTorch 模型
10.2 使用 PyTorch Simulator
10.3 加载和修改生成模型
10.4 使用生成模型重建 RKNN
10.5 量化参数存储与替换
10.6 示例与注意事项
11 RKQuantizer
11.1 接口说明
11.2 校准数据集准备
11.3 AWQ 量化配置
11.4 混合精度量化
11.5 GRQ 量化显存占用说明

1 环境说明

1.1 支持芯片

RKNN3-Toolkit 目前支持两大类芯片的模型转换，分别为**集成NPU的Soc芯片**与**协处理器芯片**。

- **集成NPU的Soc芯片**，目前支持 RK3572。
- **协处理器芯片**，目前支持 RK1820, RK1828。

不同类型芯片的在 RKNN3-Toolkit 使用上存在以下差异

功能	涉及 API 或参数	集成NPU的Soc芯片	协处理器芯片
连板功能，如连板推理、精度分析、内存占用分析、逐层耗时分析等功能	**rknn.init_runtime**	支持。设备连接上，RKNN3-Toolkit 通过 adb 接口与开发板进行调试通信。	支持。设备连接上，协处理器通过 PCIE/USB/网络 与指定主控芯片连接，RKNN3-Toolkit 通过 USB adb 或 网络 adb(abd connect）与主控芯片开发板连接、通过主控芯片开发板上的 server 程序与协处理器芯片进行调试通信。
LLM模型，将旋转位置编码保存于 host 端	rknn.config 中 llm_config 子项参数 attention_config 的次级参数 **position_embeddings_host_storage**	不支持	支持。将旋转位置编码保存于 host 端，通过牺牲每次推理的传输损耗，减少协处理器芯片上的内存占用
LLM模型，将 lm_head矩阵乘法移动至指定主控芯片处理	rknn.config 中 llm_config 子项参数 llm_head 的次级参数 **device**	不支持	支持。llm_head 的内存占用大小通常为 vocab_size x hidden_size x 量化系数。将其移动至主控芯片上处理可以减少协处理器上的内存占用，但可能会增大推理耗时。

注：表格中的**指定主控芯片**目前特指 RK3588/RK3576.

1.2 系统依赖说明

使用RKNN3-Toolkit时需要满足以下运行环境要求：

操作系统版本	Ubuntu22.04（x64）	Ubuntu24.04（x64）	macOS 13+（Apple Silicon）
Python版本	3.10	3.12	3.10 / 3.12

注：

1. 具体python库依赖详见requirements*.txt，主要依赖包括：
    - Python >= 3.10
    - numpy >= 2.0
    - onnx >= 1.16
    - onnxruntime >= 1.22.0

- torch >= 2.2
- protobuf >= 4.21.6, <= 6.33.6
- psutil >= 5.9.0
- scipy >= 1.12.0
- opencv-python >= 4.8
- pillow, sentencepiece, safetensors, datasets, tqdm, tabulate, pandas, prettytable, ruamel.yaml, fast-histogram

2. 本文档主要以Ubuntu 22.04 / Python3.10为例进行说明
3. macOS Apple Silicon 仅支持离线 ONNX 和现有 LLM 转换、build 及导出；不支持板端 runtime、设备查询、 NTP/ADB 传输、性能评估和内存评估，也不提供 Intel Mac 或 Universal2 Wheel。 请使用 `requirements_cp310.txt` 或 `requirements_cp312.txt`；macOS 与同 Python 版本的 Linux 使用相同依赖范围，包括 SciPy 和 onnxruntime >= 1.22.0。

1.3 适用的深度学习框架

RKNN3-Toolkit当前版本仅支持ONNX框架。Caffe、TensorFlow、TensorFlow Lite、DarkNet、PyTorch等框架自V1.1.0起已不再支持。

注：

1. ONNX release version和opset version、IR version之间的关系参考onnxruntime官网说明：https://github.com/microsoft/onnxruntime/blob/v1.6.0/docs/Versioning.md

2 RKNN3-Toolkit 调度流程说明

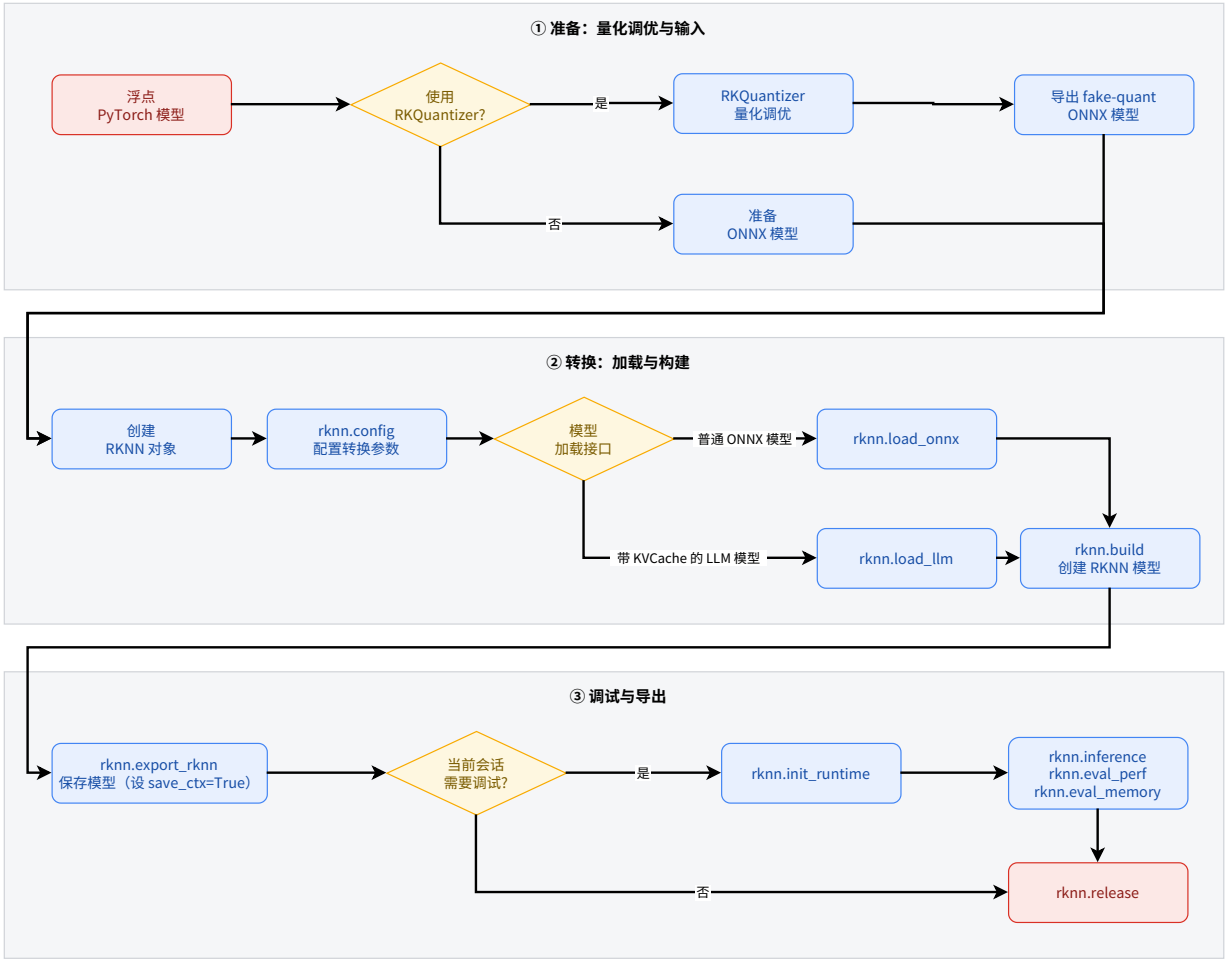
本章说明 RKNN3-Toolkit 的常用调用顺序。模型转换、重建和基于上下文加载的调试为三条相互关联但用途不同的链路：首次转换使用主链路；仅需调整已构建 LLM 模型的部分配置时使用 rebuild 链路；需要重新加载已导出模型并进行调试时使用 load_ctx 链路。

2.1 主链路：模型转换、导出与可选调试

首次转换模型时，先创建 `RKNN` 对象并调用 `config` 配置目标平台、量化和 LLM 等转换参数；然后根据模型类型调用 `load_onnx` 或 `load_llm` 加载模型，调用 `build` 构建，并通过 `export_rknn` 导出 RKNN 模型及其权重文件。需要生成可编辑 PyTorch 模型时，在 `config` 中设置 `generate_torch_model=True`；`export_rknn` 会同时在 build cache 中生成 ONNX→PyTorch 产物。

对于需要在 Toolkit 中进一步调试、进行精度分析或在后续会话中重新加载并调试的场景，导出时应设置 `save_ctx=True` 保存上下文信息。构建完成后也可在当前 `RKNN` 对象上调用 `init_runtime` 初始化运行环境，再调用 `inference`、`eval_perf` 或 `eval_memory` 进行推理、性能或内存调试。

在导出 ONNX 之前，可选用 `RKQuantizer` 对 PyTorch 模型的 Linear 权重进行量化调优；该步骤输出 fake-quant 模型，用户需自行将其导出为 ONNX 后再进入 RKNN 转换流程。各链路完成导出或调试后，应调用 `rknn.release` 释放 `RKNN` 对象占用的资源。

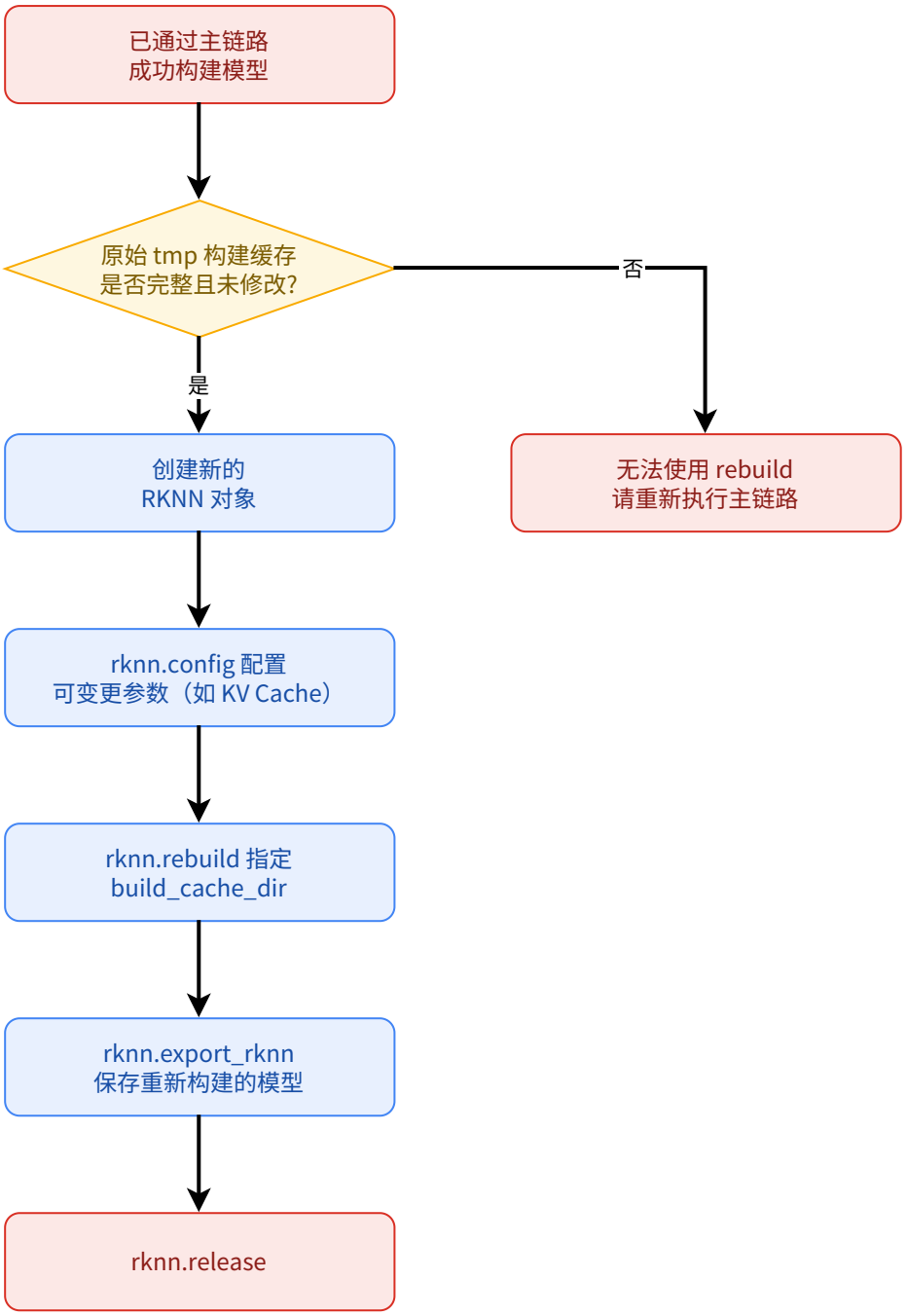


> 注意：若后续需要通过 `load_rknn(load_ctx=True)` 重新加载模型并进行调试，请在此链路的 `export_rknn` 调用中设置 `save_ctx=True`。

2.2 rebuild 链路：基于构建缓存重新构建模型

`rebuild` 用于在已完成主链路构建后，基于 `build` 阶段生成的 `tmp` 构建缓存重新构建模型。该链路的前提是原始主链路已成功执行，且对应的 `tmp` 目录完整、未被修改或删除。

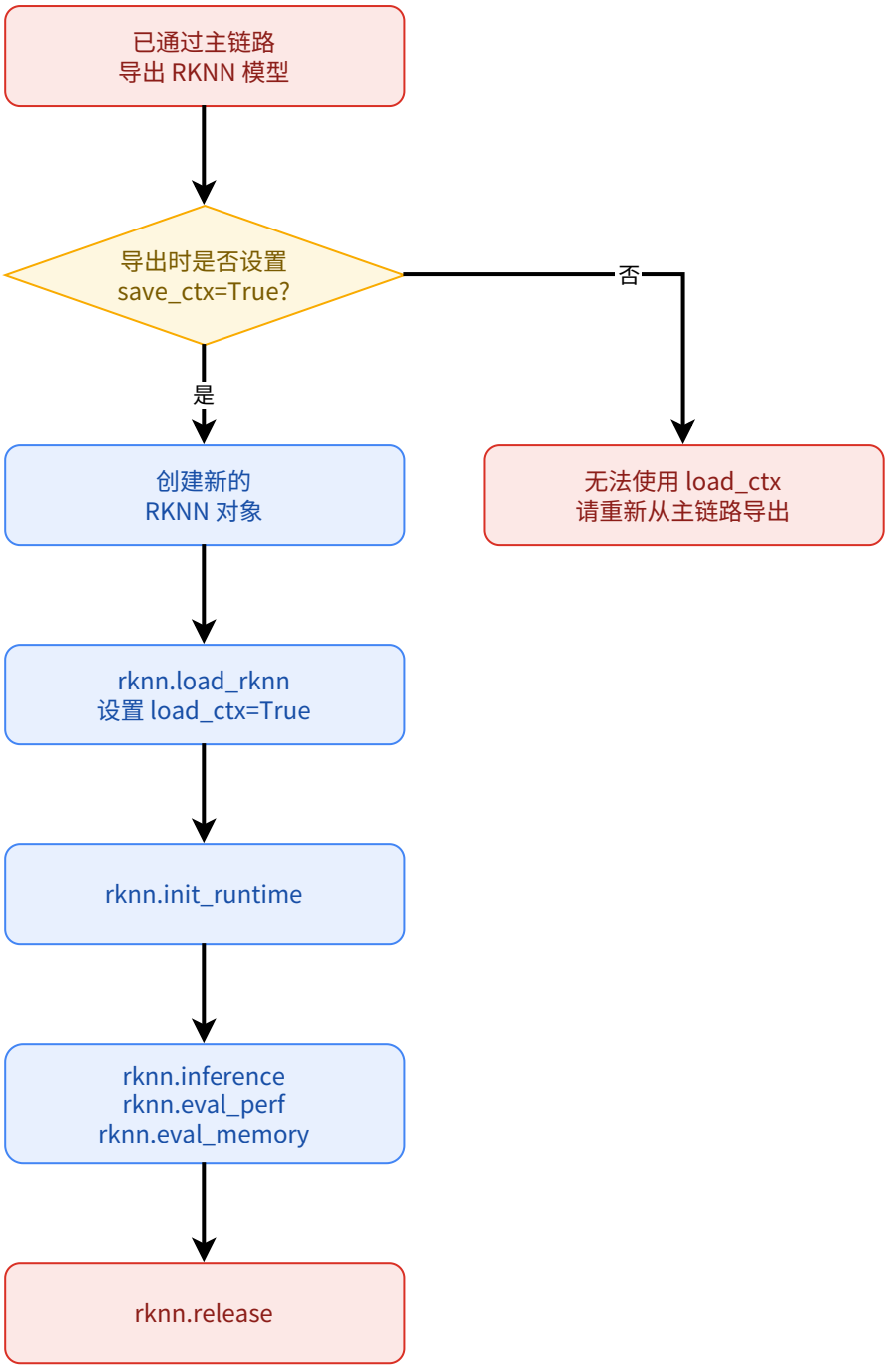
创建新的 `RKNN` 对象后，可通过 `config` 调整 rebuild 支持变更的参数，例如 `profile_mode` 及 `llm_config` 中与 KV Cache 相关的配置；随后调用 `rebuild(build_cache_dir=...)` 并指定原始构建缓存目录，最后使用 `export_rknn` 导出重新构建的模型。包含 ONNX→PyTorch 产物的 cache 还支持读取修改后的生成模型，或通过 `quant_json` 替换兼容量化参数。



2.3 load_ctx 链路：加载上下文并调试已导出模型

`load_ctx` 链路用于在新的 Toolkit 会话中重新加载已导出的 RKNN 模型并执行推理、性能或内存调试。其前提是模型已通过主链路导出，且当时 `export_rknn` 已设置 `save_ctx=True`，从而生成对应的上下文信息文件。

创建 `RKNN` 对象后，调用 `load_rknn` 并设置 `load_ctx=True` 加载模型、权重和上下文信息；随后调用 `init_runtime` 初始化运行环境，即可使用 `inference`、`eval_perf` 或 `eval_memory` 进行调试。未加载上下文时，模型仍可用于板端推理，但不能用于模拟器或精度分析等依赖上下文的功能。



2.4 RKNN 对象与资源生命周期

使用 RKNN3-Toolkit 的 API 前，需要先调用 `RKNN()` 创建 RKNN 对象；完成模型转换、调试或评估后，应调用 `release()` 释放该对象占用的资源。初始化时可通过 `verbose` 控制详细日志输出，并可通过 `verbose_file` 将日志写入指定文件。

举例如下：

```
# 打印详细日志
rknn = RKNN(verbose=True, verbose_file=None)

...

rknn.release()
```

3 模型配置接口

在构建RKNN模型之前，需要先对模型进行通道均值、量化图片RGB2BGR转换、量化类型等的配置，这些操作可以通过config接口进行配置。

API	config
描述	设置模型转换参数。
参数	**mean_values**：输入的均值。参数格式是一个列表，列表中包含一个或多个均值子列表，多输入模型对应多个子列表，每个子列表的长度与该输入的通道数一致，例如[[128,128,128]]，表示一个输入的三个通道的值减去128。 默认值为None，表示所有的mean值为0。
	std_values：输入的归一化值。参数格式是一个列表，列表中包含一个或多个归一化值子列表，多输入模型对应多个子列表，每个子列表的长度与该输入的通道数一致，例如[[128,128,128]]，表示设置一个输入的三个通道的值减去均值后再除以128。 默认值为None，表示所有的std值为1。
	quant_img_RGB2BGR：表示在加载量化图像时是否需要先做RGB2BGR的操作。如果有多个输入，则用列表包含起来，如[True, True, False]。默认值为False。 该配置只对量化图像格式为jpg/jpeg/png/bmp有效，npy格式读取时会忽略该配置，因此当模型输入为BGR时，npy也需要为BGR格式。 **该配置仅用于在量化阶段（build接口）读取量化图像或量化精度分析（inference接口中设置accuracy_analysis=True），并不会保存在最终的RKNN模型中，因此如果模型的输入为BGR，则在调用RKNN3-Toolkit的inference或C-API的run函数之前，需要保证传入的图像数据也为BGR格式。**
	quantized_dtype：量化类型，目前支持的量化类型有w4a16、w4abf16、w6a16、w8a16、w16a16、w8a8。其中w表示权重量化，a表示激活值量化。默认值为w16a16。 **- w16a16**：权重与激活值均为float16精度，相当于无量化。 **- w4a16**：权重为4bit非对称量化精度，激活值为16bit浮点精度。 **- w4abf16**：权重为4bit非对称量化精度，激活值为bfloat16精度。 **- w6a16**：权重为6bit非对称量化精度，激活值为16bit浮点精度。 **- w8a16**：权重为8bit非对称量化精度，激活值为16bit浮点精度。 **- w8a8**：权重为8bit非对称量化精度，激活值为8bit非对称量化精度。
	quantized_algorithm：计算每一层的量化参数时采用的量化算法，目前支持的量化算法有：normal，mmse，kl_divergence。默认值为normal。 **normal**量化算法的特点是速度较快，推荐量化数据集量一般为20-100张左右，更多的数据集量下精度未必会有进一步提升。 **mmse**量化算法由于采用暴力迭代的方式，速度较慢，但通常会比normal具有更高的精度，推荐量化数据集量一般为20-50张左右，用户也可以根据量化时间长短对量化数据集量进行适当增减。 **kl_divergence**量化算法所用时间会比normal多一些，但比mmse会少很多，在某些场景下（feature分布不均匀时）可以得到较好的改善效果，推荐量化数据集量一般为20-100张左右。
	quantized_method：目前支持layer，channel或者group{SIZE}。默认值为None, 当 quantized_dtype 不是 w16a16 时，则必须配置有效参数。 - layer：每层的weight只有一套量化参数； - channel：每层的weight的每个输出通道都有一套量化参数，通常情况下channel会比layer精度更高。 - group{SIZE}：在channel的基础上，每个输出通道的weight在输入通道上又按{SIZE}细分为多个小组，每个小组有各自的一套量化参数，通常情况下group{SIZE}会比channel精度更高。{SIZE}是32到256之间的32的倍数值，如group32或group128。目前group{SIZE}只在quantized_dtype为w4a16/w4abf16/w6a16/w8a16下有效。分组量化当前只支持MatMul/Linear类型的op，分组量化目前不支持卷积，卷积会维持 w16a16 的精度。

API	config
	optimization_level：模型优化等级。默认值为3。 通过修改模型优化等级，可以关掉部分或全部模型转换过程中使用到的优化规则。该参数的默认值为3，打开所有优化选项。值为2或1时关闭一部分可能会对部分模型精度产生影响的优化选项，值为0时关闭所有优化选项。
	target_platform：指定 RKNN 模型基于哪个目标芯片平台生成，支持芯片请参考 1.1 章节。该参数对大小写不敏感。默认值为None。
	dynamic_input：用于根据用户指定的多组输入shape，来模拟动态输入的功能。格式为[[input0_shapeA, input1_shapeA, ...], [input0_shapeB, input1_shapeB, ...], ...]。 默认值为None。 假设原始模型只有一个输入，shape为[1,3,224,224]，或者原始模型的输入shape本身就是动态的，如shape为[1,3,height,width]或[1,3,-1,-1]，但部署的时候，需要该模型支持3种不同的输入shape，如[1,3,224,224], [1,3,192,192]和[1,3,160,160]，此时可以设置dynamic_input=[[[1,3,224,224]], [[1,3,192,192]], [[1,3,160,160]]]，转换成RKNN模型后进行推理时，需传入对应shape的输入数据。 注： **1. 需要原始模型本身支持动态输入才可开启此功能，否则会报错。** **2. 如果原始模型输入shape本身就是动态的，则只有动态的轴可以设置不同的值。**
	core_num：模型需要用到的NPU核心数。RK1820的NPU核心数范围为1到8，默认值为0，表示自动。 注： 1. Transformer类模型可以根据需要设置使用的NPU核心数，其他模型（如CNN模型）建议设为1。 2. 自动模式下，在使用 load_llm 接口加载模型时会调整为使用8个NPU核心，如果通过 load_onnx 等其余接口则会调整为使用1个NPU核心。 3. RK3572平台为单核NPU，仅支持设置为1。
	rknn_build_parallelism：编译 rknn 模型的并行线程数，默认为 -1，表示自动配置，自动配置时不会使用超过32线程。
	input_attrs： 模型部署时的输入属性，默认值为{}。格式为：{input_name: {attr_name: attr_value, ...}, ...}, 例如：{'input_name': {'dtype': 'float32', 'layout': 'NCHW', 'shape': [1, 3, 224, 224]}, ...} 其中attr_name的可选项为dtype, layout, shape, stride等。
	output_attrs： 模型部署时的输出属性，默认值为{}。格式为：{output_name: {attr_name: attr_value, ...}, ...}, 例如：{'output_name': {'dtype': 'float32', 'layout': 'NCHW', 'shape': [1, 1000]}, ...} 其中attr_name的可选项为dtype, layout, shape, stride等。
	generate_torch_model：是否在 `export_rknn` 时根据当前 build cache 生成可编辑的 PyTorch 模型、独立权重和回导 ONNX。默认值为 False。生成产物位于 `<build_cache_dir>/torch_model`，用于 PyTorch Simulator、模型调试和 `rebuild`。
	llm_config：配置 LLM 相关优化项，配置格式为字典，详细配置说明见3.3小节。配置后需要调用 rknn.load_llm 接口加载模型才会生效。
返回值	无。

举例如下：

```
# model config
rknn.config(mean_values=[[103.94, 116.78, 123.68]],
            std_values=[[58.82, 58.82, 58.82]],
            target_platform='rk1820')
```

3.1 实验参数

以下参数为实验性功能，后续版本可能调整或移除：

参数	说明
custom_string	允许用户配置字符串内容并保存于 RKNN 模型中，在后续的 CAPI 接口中查询并获取，**目前暂不支持**。默认为 None
cvt_conv_streaming	填入一组卷积OP的输出tensor名称，将指定卷积转为流式卷积。流式卷积 Cache 将由 NPU 内部管理，推理性能优于外部管理 Cache 的模式，支持1D/2D卷积，转换成功时模型会额外生成一个 Conv_streaming_Tc 输入, 用于控制当前推理的有效长度。该参数默认为空列表。 - 目前只支持在单个维度上进行流式推理，2D卷积转换时会自动识别流式作用轴位于 Height 或 Width 上，两个轴同时满足流式要求时默认取 Width 轴作为流式轴。对于2D卷积, 建议用户设计模型时选取 Width 作为流式作用轴以获得较好的推理性能。流式轴要求对应的 (kernel-1)*dilation <= 左pad、且右pad参数为0。目前不支持手动指定流式轴的位置。 - 若模型存在多组不同规格的流式卷积，如模型存在 upsample 模块时，前后流式卷积的尺度可能是不一致的。此时允许填入多组卷积OP的输出tensor名称，并生成多个 Conv_streaming_Tc_{N} 输入。 - 对于 onnx 模型带有显性流式卷积Cache输入输出的模型, 用户需要在使用 load_onnx 接口时指定有效输出、移除流式卷积输出，并在 config 中配置 input_initial_value 参数将对应的流式卷积Cache输入配置为全零输入。对于使用 load_llm 接口的用户，目前需要用户手动在外部将流式卷积Cache输出从 onnx.model.graph.output 中移除，这种使用方式可以参考 Qwen3.5 模型示例。 - 单组示例：cvt_conv_streaming=['Conv0_output0', 'Conv1_output0'] - 多组示例：cvt_conv_streaming=[['Group0_conv0_output0', 'Group0_conv1_output0'], ['Group1_conv0_output0', ...], ...]
disable_logN_rcclReduce	配置 True 时关闭多核并行处理 rcclReduce 通信操作，影响 rcclReduce 通信算子的耗时。默认为 False
distribute_full_search_space	模型拆分搜索空间。配置为True时，多核拆分步骤会尝试不同的 op 检索顺序，从中找到性能最优的解。默认为 False。
distribute_strategy	模型拆分策略。目前支持'less_subgraph', 'best_perf', 'less_mem'。拆分多核执行图时, 'less_subgraph' 表示尽可能减少子图数量； 'less_mem' 表示尽可能减少常量内存占用； 'best_perf' 表示以性能优先形式进行拆分，允许权重被拷贝到多核上, 例如允许卷积沿 hw 方向切分。 默认值为'less_mem'。
float_dtype	配置 rknn 的 float 类型，目前支持 'float16'、'bfloat16'。默认为 'float16'。
inputs_yuv_fmt	为指定输入添加 YUV->RGB 变换算子, 支持 'nv12','nv21' 类型的YUV输入。配置为同输入等长的列表, 不需要转换的输入配置为None。 例如模型有两个输入, 第二个需要插入 'nv12' 的转换, 配置 [None, 'nv12']。 默认为 None。**该功能暂不支持**。

参数	说明
input_initial_value	为模型的输入配置初始化常量，减少模型推理时的输入传输开销，支持动态 shape 使用情景下为不同 shape 分配不同的输入初始化常量。填入格式为2级嵌套list，第二级 list 的元素个数需与 onnx 模型输入个数一致。允许填入的类型为 None 和 np.ndarray，None 表示对应输入不需要初始常量，配置 np.ndarray 时需使用与 onnx 输入一致的数据类型。 - 与动态 shape 一起使用时，请注意多组参数的顺序需要与动态 shape 多组 shape 顺序保持一致。默认为 None。 - 无动态 shape 示例：input_initial_value=[[None, np.arange(16, dtype=np.int32)]]。 - 动态 shape 示例：dynamic_input=[[[1,3,192,192], [1,16]], [[1,3,384,384], [1,64]]], input_initial_value=[[None, np.arange(16, dtype=np.int32)], [None, np.arange(64, dtype=np.int32)]]
merge_mean_std_to_conv	将输入 mean,std 参数融入到第一个卷积里面，推理时则不需要额外的归一化操作，可以提升模型的整体推理性能。若第一个卷积带有 pads 参数，此操作只会融合std，不会融合mean参数，避免hw维度边缘的数值出现偏差。 开启时对所有输入均生效。 默认为 False。
model_pruning	对模型进行无损剪枝, 生效OP为卷积，开启时尝试剔除全零的输入通道或全零的输出通道，可以减小转换后RKNN模型的大小和计算量。通常需要训练、后训练去构造卷积的通道稀疏特性，未经处理的模型通常无此特性。默认值为False。
op_quantized_dtype	指定 op 的量化类型。默认为空字典 - 目前只支持指定 Gemm/Matmul/Linear 类型 op 的量化类型 - 指定的量化类型只支持 'w4a16', 'w6a16', 'w8a16', 'w16a16' - 输入格式为字典，key 可以是 node 名、node 输出或常量名。如 {'Conv': 'w8a16'}, {'Conv_output0': 'w8a16'}, {'onnx::weight123': 'w8a16'}
profile_mode	配置后转出的 rknn 会保留额外的中间信息, 使用精度分析功能需开启此功能。默认为 False。
sdpa_qk_scale	为 Attention 算子中的 Q,K 应用缩放因子，用于避免 exSDPA 计算过程中可能的溢出问题。参数格式为字典，key 为 exSDPA 算子名称，value 为缩放系数列表， 例如 {'exSDPA_op_name' : [float(1/50), float(1/50)]}。 默认为空字典，表示不进行额外缩放。
shard_map	指定具体op的多核切分方式。**目前暂不支持**。默认为空字典。
subgraph_dtype_target	将模型中指定片段的推理精度类型配置为目标类型。该参数目前仅在模型开启量化时生效，且目前只支持指定对应片段使用 w16a16（等同于 Float16）进行推理。 参数为两级list，例如 [[subgraph0_inputs, subgraph0_outputs, dtype], [subgraph1_inputs, subgraph1_outputs, dtype], ...]，其中 subgraph_inputs 为包含片段一个或多个输入 tensor 名称的 list，subgraph_outputs 为包含片段一个或多个输出 tensor 名称的 list，tensor 名称可从精度分析功能生成的表格中获取。dtype 表示推理精度类型。 注1: 特别的，当 subgraph_inputs、subgraph_outputs 为某个op的输入输出，此时相当于指定该 op 使用分配的推理精度类型。 注2: 目前不强制要求 subgraph_inputs、subgraph_outputs 必须包含对应片段的所有输入输出，此时片段中的部分 op 会使用对应的类型，判断逻辑为 op 的输入与 subgraph_inputs 有推理关联、同时 op 的输出与 subgraph_outputs 有推理关联，此时会将该 op 配置为指定的推理精度类型。

参数	说明
sparse_infer	模型里面的卷积权重符合稀疏化要求时, npu 开启稀疏化推理。**目前暂不支持**。默认为False。
simulate_lut	开启后模拟器推理激活函数算子时，会模拟 npu 行为使用查表实现，与 npu 的精度一致性会提高，模拟器推理性能会下降。默认为 False。

3.2 已弃置参数

以下参数已弃置:

参数	说明
attn_nodes_config	功能已弃置。
auto_hybrid_cos_thresh	功能已弃置。
auto_hybrid_euc_thresh	功能已弃置。
kvcache_dtype	请使用 `llm_config` 参数 `attention_config` 中的 `kvcache_dtype` 替代。
kvcache_store_method	内部根据 `kvcache_dtype` 自动判断, 不再需要配置。
kvcache_group_size	内部根据 `kvcache_dtype` 自动判断, 不再需要配置。
kvcache_residual_depth	内部根据 `kvcache_dtype` 自动判断, 不再需要配置。
max_ctx_len	请使用 `llm_config` 参数 `attention_config` 中的 `kvcache_buffer_len` 替代。
max_position_embeddings	请使用 `llm_config` 参数 `attention_config` 中的 `max_position_embeddings` 替代。
position_embeddings_host_storage	请使用 `llm_config` 参数 `attention_config` 中的 `position_embeddings_host_storage` 替代。
remove_reshape	功能已弃置。
single_core_mode	功能已弃置。

3.3 llm_config 配置项

针对 LLM 模型， `rknn.config` 新增了 `llm_config` 参数，LLM config 主要有4个字典子项，分别为 `attention_config` , `llm_head` , `vocab` , `keep_one_logit` 。

---

attention_config：用于配置 KV Cache 缓存机制、配置 Attention op 的推理机制，接受类型为列表，列表中允许存在多个字典指定多种 Attention 结构的具体参数。Attention_config 的子字典定义如下：

配置	功能	数据类型:默认值	备注
mask_name	指定 mask 名称，在后续流程中 Attention op 会通过识别 mask 输入来确定该 Attention op 所属的相关配置。 **该参数不可缺省!**	String: 'attention_mask'	不同类型的 Attention 结构不允许共享同一个 mask 输入

配置	功能	数据类型:默认值	备注
kvcache_buffer_len	指定 KV Cache 缓存长度。推理时若上下文长度超过设定长度时，遵循先进先出的原则，进行循环覆写。配置数值越大时，KV Cache占用内存越大，长文本任务的表现理论上会更好。	Int:1024	数值大于512时要求512对齐；不大于512时要求与 `kvcache_group_size` 对齐。
tokens_per_page	分页 KV Cache 的每页 token 数；设为 `None` 使用原有连续 KV Cache。	Int:512	非正数或 SlidingAttention 会改为 `kvcache_buffer_len`；最终值必须是 `max(MacC, MacK)` 的整数倍。
request_num	分页 Attention 的请求数。	Int:1	必须为正整数；多请求当前仅支持解码，每个请求的 `Tc` 为 0 或 1。
kvcache_dtype	指定上下文保存及计算数据类型，影响推理性能及缓存内存占用大小。 内存占用上, Float16 > Int8_to_F16 > Int4_to_F16	String: 'Float16'	支持 Float16, Int8_to_F16, Int4_to_F16
attention_type	指定 Attention 类型	String: 'FullAttention'	支持 FullAttention, SlidingAttention.
sliding_window_size	指定 Sliding Attention 的滑窗长度。当 attention 类型为 FullAttention 时，此参数不生效	Int: -1	/
position_name	指定 position id 输入的名字，用于提取位置编码特征值。对于早期的模型，可能不存在位置编码，此时允许缺省，配置为 None.	String: 'position_ids'	不同类型的 Attention 结构不允许共享同一个 position_ids 输入
max_position_embeddings	指定位置编码的最大长度。推理时若上下文长度超过该设定长度时，会抛出错误。	Int: 8192	当 `position_name` 非 None 且 `kvcache_buffer_len` 大于该值时，会自动将该值更新为 `kvcache_buffer_len`。

配置	功能	数据类型:默认值	备注
position_embeddings_host_storage	指定位置编码（RoPE）的存储位置。配置为 `False` 时，位置编码常驻于设备内存；配置为 `True` 时，位置编码存储在 Host 端内存中。	Bool：False	/
mrope_type	当 LLM 模型为特殊的 mrope 类型时，可以配置并开启 mrope 功能。目前支持 ['Qwen2.5-VL', 'Qwen3-VL', 'Qwen3.5'] 类型。**注意**，开启 mrope 时必须配置 position_name，且这个 position_name 是单维的输入，开启 mrope 功能后 toolkit 会自动生成一个 [seq_len, 3] 维度的输入，用于填充 mrope 的维度信息	Strings: None	/
mrope_section	配置 mrope section 信息。对于 Qwen2.5-VL、Qwen3-VL、Qwen3.5 模型，可在模型的配置 config.json 中找到对应的配置值。	list: None	/
mrope_new_id_name	配置 mrope 新生成的 position 名称。对于当前支持的 ['Qwen2.5-VL', 'Qwen3-VL'，'Qwen3.5']，会生成维度为 [seq_len, 3] 的新输入。	Strings: None	/

llm_head：用于配置并开启 LLM 模型 Head 层的优化行为。（Head层通常为单一一个参数量较大的Linear层）。**暂不支持多个 LLM Head 层**，后续会完善并支持。

配置名	功能	数据类型:默认值	备注
name	配置 llm head 层的权重名称	String: auto	目前仅支持 auto，暂不支持指定
device	配置 llm head 层的计算设备	String: rk1820	支持 rk1820, rk3588, rk3576. 指定为 rk3588/rk3576 会将 llm_head 的矩阵计算分发到作为主控的 rk3588/rk3576 的 NPU 上进行计算, 节省协处理器上的内存资源, 但可能会增加计算耗时

配置名	功能	数据类型: 默认值	备注
quantized_dtype	配置 llm head 层的量化类型。llm head 层规模较大，采用量化可以有效降低内存占用大小	String: w6a16	支持 w16a16, w4a16, w6a16, w8a8. 当平台为 rk3576 或 rk3588 时，会被强制更改为 w8a8.
quantized_method	配置 llm head 层的量化方式。	String: group32	支持 layer, channel, group{SIZE}. 当平台为 rk3576 或 rk3588 时，会被强制更改为 channel.

vocab：用于配置并开启 LLM 模型对字典层的优化行为。

配置名	功能	数据类型:默认值	备注
index_name	配置 vocab 层的 index 输入名称	String: auto	目前仅支持 auto，暂不支持指定
tie_embeddings	配置 vocab 层的 tie embedding 参数，tie_embeddings 为 True 时意味着 vocab 和 llm_head 共享同一个数组。开启后则不再需要额外保存 vocab，但由于 llm_head 存在量化行为，反量化为 float16 的 embedding 时会降低推理性能。	Bool: False	/

keep_one_logit：用于配置并开启对输出的指定轴进行取单值行为（插入gather op）。对于大多数 LLM 模型，在输出采样时只需要对最后一个有效token输出进行采样即可。以输入token 序列长度为 100 为例，则输出的第100个序列为有效输出，其他输出均被舍弃。故可以将这个舍弃行为提前到 llm_head 前面，减少 llm head 线性 Linear 层的计算量、减少计算过程中所需要的内存占用。

配置名	功能	数据类型:默认值	备注
output_name	指定输出名称	String: None	为None 时则不生效
idx_name	生成 gather indices 输入的名称，并加入到模型的输入	String: num_logits_to_keep	/
axis	指定 gather 生效轴	Int: None	/

举例如下：

```
from rknn.api import RKNN, DEFAULT_RKNN_LLM_CONFIG
rknn = RKNN(verbose=True)
my_config = DEFAULT_RKNN_LLM_CONFIG.copy()
rknn.config(target_platform='rk1820', llm_config=my_config)

# default config is
DEFAULT_RKNN_LLM_CONFIG = {
'attention_config': [{
    'mask_name'              : 'attention_mask',
    'kvcache_buffer_len'     : 1024,
    'tokens_per_page'        : 512,
    'request_num'            : 1,
    'kvcache_write_policy'   : 'WRITE_THROUGH',
    'kvcache_dtype'          : 'Float16',
    'attention_type'         : 'FullAttention',
    'sliding_window_size'    : -1,
    'position_name'          : 'position_ids',
    'max_position_embeddings': 8192,
    'position_embeddings_host_storage': False,
    'mrope_type'             : None,
    'mrope_section'          : None,
    'mrope_new_id_name'      : None,
}],
'llm_head':[{
    'name'                  : 'auto',
    'device'                : 'RK1820',
    'quantized_dtype'       : 'w6a16',
    'quantized_method'      : 'group32',
}],
'vocab':[{
    'index_name'            : 'auto',
    'tie_embeddings'         : False,
}],
'keep_one_logit': [{
    'output_name': None,
    'idx_name'   : 'num_logits_to_keep',
    'axis'       : None,
}],
}
```

4 模型加载接口

RKNN3-Toolkit当前版本仅支持ONNX模型的加载转换，以及LLM模型的专用加载接口。Caffe、TensorFlow、TensorFlow Lite、DarkNet、PyTorch等模型的加载接口自V1.1.0起已移除。

4.1 ONNX模型加载

API	load_onnx
描述	加载ONNX模型。
参数	**model**：ONNX模型文件（.onnx后缀）路径。
	inputs：模型输入节点（tensor名），支持多个输入节点，所有输入节点名放在一个列表中。默认值为None，表示从模型里获取。
	input_size_list：每个输入节点对应的shape，所有输入shape放在一个列表中。如inputs有设置，则input_size_list也需要被设置。默认值为None。
	input_initial_val：设置模型输入的初始值，格式为ndarray的列表。默认值为None。 主要用于将某些输入固化为常量，对于不需要固化为常量的输入可以设为None，如[None, np.array([1])]。
	outputs：模型的输出节点（tensor名），支持多个输出节点，所有输出节点名放在一个列表中。默认值为None，表示从模型里获取。
返回值	0：导入成功。
	-1：导入失败。

举例如下：

```
# 从当前目录加载arcface模型
ret = rknn.load_onnx(model='./arcface.onnx')
```

4.2 LLM模型加载接口

API	load_llm
描述	加载LLM模型。 LLM模型参考rkllm3_model_zoo工程的示例导出。（例如rkllm3_model_zoo/examples/Qwen2_5/python/export_llm.py）
参数	**model**：LLM模型文件（.onnx后缀）路径。
	config：LLM模型的配置文件路径。
	seq_lens：LLM模型输入sequence length列表，默认为[1,128]。
返回值	0：导入成功。
	-1：导入失败。

举例如下：

```
# 从当前目录加载Qwen2.5-0.5B模型
ret = rknn.load_llm(model='./Qwen2.5-0.5B.onnx',
                    config='./Qwen2.5-0.5B.pkl')
```

5 模型构建和量化接口

5.1 构建RKNN模型

API	build
描述	构建RKNN模型。
参数	**do_quantization**：是否对模型进行量化。默认值为True。
	dataset：用于量化校正的数据集。目前支持文本文件格式，用户可以把用于校正的图片（jpg或png格式）或npy文件路径放到一个.txt文件中。文本文件里每一行一条路径信息。如： a.jpg b.jpg 或 a.npy b.npy 如有多个输入，则每个输入对应的文件用空格隔开，如： a.jpg a2.jpg b.jpg b2.jpg 或 a.npy a2.npy b.npy b2.npy 注：量化图片建议选择与预测场景较吻合的图片。
返回值	0：构建成功。
	-1：构建失败。

举例如下：

```
# 构建RKNN模型，并且做量化
ret = rknn.build(do_quantization=True, dataset='./dataset.txt')
```

5.2 重建RKNN模型

API	rebuild
描述	重建RKNN模型。重建前，允许调用 config 变更导出的 RKNN 模型行为。目前 config 参数中的 **profile_mode** 以及 llm_config 参数支持在 rebuild 阶段进行变更，其中 llm_config 的 attention_config 中的 **kvcache_buffer_len**、**kvcache_dtype**、**kvcache_store_method**、**kvcache_group_size**、**kvcache_residual_depth** 、**max_position_embeddings** 支持在重建阶段变更。对于启用 `generate_torch_model` 的 build cache， `rebuild` 还可以读取修改后的 `torch_model/generated.py` 、权重和量化参数；图结构发生变化时会重新校准并执行完整重建。**请注意**，rebuild 用法目前不支持后续调用连板功能，如连板推理、精度分析；也不支持后续使用 export_rknn 时配置 save_ctx=True。
参数	**build_cache_dir**：指定在 rknn.build 阶段中位于当前执行目录下产生的 tmp 文件夹。该参数为必填参数。
	strict_check： 是否严格检查。部分情况下 tmp 目录有可能因为微小变更、或者在 rebuild 过程中被中断，此时有可能导致检查无法通过，此时需要配置 strict_check 为 False。默认为 True
	quant_json：指定替换量化参数的 manifest JSON。仅对包含 ONNX→PyTorch 产物的 build cache 生效；不指定时使用原 build cache 的量化参数。图结构不变时可直接替换兼容的量化参数，详见第 10 章。默认值为 None。
	dataset：当 `generated.py` 修改导致量化图结构发生变化时，用于重新校准的 dataset。只替换兼容量化参数且图结构不变时可省略。默认值为 None。
返回值	0：构建成功。
	-1：构建失败。

举例如下：

```
# 重建RKNN模型，并导出模型
rknn = RKNN(False)
ret = rknn.rebuild(build_cache_dir='./tmp', strict_check=True)
rknn.export_rknn('mobilenet_v1.rknn')
```

6 RKNN模型导出接口

通过本工具构建的RKNN模型通过该接口可以导出存储为RKNN模型文件，用于模型部署。

API	**export_rknn**
描述	将RKNN模型保存到指定文件中（.rknn后缀）且将模型权重保存在同一文件路径中（.weight后缀）。 如save_ctx为True，则也会保存上下文信息（.rknn.ctx后缀）。当 `config(generate_torch_model=True)` 时，该接口还会生成第 10 章所述的 PyTorch 产物；生成失败时导出返回 -1。
参数	**export_path**：导出模型文件的路径。
	save_ctx：是否导出上下文信息（配合load_rknn的load_ctx参数使用）。默认值为False。
返回值	0：导出成功。
	-1：导出失败。

举例如下：

```
# 将构建好的RKNN模型保存到当前路径的mobilenet_v1.rknn文件中，模型权重保存在当前路径的
mobilenet_v1.weight文件中
ret = rknn.export_rknn(export_path='./mobilenet_v1.rknn')
```

7 运行时初始化与推理接口

7.1 加载RKNN模型

API	load_rknn
描述	加载RKNN模型。 加载完RKNN模型后，不需要再进行模型配置、模型加载和构建RKNN模型的步骤。 当load_ctx为False时，加载后的模型仅限于连接NPU硬件进行推理或获取性能数据等，不能用于模拟器或精度分析等。 当load_ctx为True时，加载后的模型不仅可连接NPU硬件进行推理或获取性能数据等，也可用于模拟器或精度分析等。
参数	**model_path**：RKNN模型文件路径。
	weight_path：RKNN模型权重文件路径。
	lora_weight_path：LoRA权重文件路径，支持单个 `.lora_weight` 文件、包含多个 `.lora_weight` 文件的目录或路径列表。
	load_ctx：是否加载上下文信息。默认值为False。
返回值	0：加载成功。
	-1：加载失败。

举例如下：

```
# 从当前路径加载mobilenet_v1.rknn模型
ret = rknn.load_rknn(model_path='./mobilenet_v1.rknn',
weight_path='./mobilenet_v1.weight')
```

7.2 初始化运行时环境

在模型推理或性能评估之前，必须先初始化运行时环境，明确模型的运行平台（具体的目标硬件平台或软件模拟器，硬件设备的连接说明可以参考 1.1 章节）。

API	init_runtime
描述	初始化运行时环境。
参数	**target**：目标硬件平台。 默认为 None（模型在模拟器运行）；指定具体平台时，模型在对应设备运行。支持list的组合模式 ['', 'simulator']，可同时在真实设备与模拟器上执行，主要用于大语言模型多轮精度分析时在模拟器中保留键值缓存（KV Cache）。
	device_id：设备编号，当PC连接多台设备时，需通过该参数指定目标设备编号；使用协处理器时，指定的是主控的设备编号。可通过调用 list_devices 接口查询可用设备列表。默认值为None。
	core_mask：模型在板端（硬件）推理时使用的 NPU 核心掩码，进行板端推理时指定该参数，支持从0x1到0xff的掩码值。默认值为-1。
	custom_op_plugins：自定义算子插件共享库（.so）的路径。该插件将在模型初始化后注册到运行时上下文中。若已通过 `reg_custom_op` 接口指定了插件路径，则无需在此重复传入。默认值为None。 **注意**：如果模型通过 `reg_custom_op` 注册了自定义算子但未在该接口指定插件路径，则**必须**在 `init_runtime` 中通过 `custom_op_plugins` 传入。

API	init_runtime
	simulator_backend：软件模拟器后端。支持 `'onnx'`（默认）和 `'pytorch'`；设置为 `'pytorch'` 时使用 ONNX→PyTorch 生成产物推理。仅在 `target=None` 或 `target='simulator'` 时生效。
	pytorch_artifact_dir：包含 `generated.py` 和 `generated_weights.pth` 的 `torch_model` 目录。使用 PyTorch Simulator 时必填。
	pytorch_quant_json：可选的量化参数 manifest JSON，用于在 PyTorch Simulator 中覆盖生成模型的量化权重及 scale/zero-point。
返回值	0：初始化运行时环境成功。
	-1：初始化运行时环境失败。

注（连接方式）：当前仅支持 PC 通过 ADB 模式连接设备，连接方式分为以下两种（详见 1.1）：

- **协处理器芯片**：PC 通过 ADB 连接**指定主控**（其 SoC 必须为 RK3576 或 RK3588）；主控再通过 PCIe 或 USB 连接**协处理器芯片**（RK1820/RK1828）。此时 `target` 填算力芯片型号，`device_id` 填主控设备编号。
- **集成NPU的Soc芯片**：PC 通过 ADB 直接连接，此时 `target` 填算力芯片型号，`device_id` 填该设备编号。

举例如下：

```
# 初始化运行时环境
ret = rknn.init_runtime(target=None)
```

7.2.1 分配和扩容分页 KV Cache

仅在分页 Attention 模型已通过 `init_runtime(target=...)` 初始化硬件运行时后使用。首次推理若未显式分配，会自动按模型完整上下文容量分配；需要控制初始内存占用时，可在首次推理前调用 `alloc_kvcache_pages(max_tokens)`。运行时必须空闲，扩容时保留已有页面内容。

API	alloc_kvcache_pages(max_tokens=None) / grow_kvcache_pages(max_tokens=None)
描述	`alloc_kvcache_pages` 首次分配并绑定分页缓存，重复调用时仅扩容；`grow_kvcache_pages` 扩容已有缓存，尚未分配时执行首次分配。仅 FullAttention 页面按 `max_tokens` 计算页数，其他缓存类型按模型要求分配。
参数	**max_tokens**：正整数，表示 FullAttention 需要覆盖的 token 容量，不得超过模型最大上下文长度；`None` 表示完整容量。
返回值	0：成功；负数：运行时错误。仅支持硬件运行时和支持分页缓存的模型及运行时。

```
rknn.init_runtime(target='rk1820')
rknn.alloc_kvcache_pages(1024)
# 在运行时空闲时扩容，原有页内容保留
rknn.grow_kvcache_pages(2048)
```

7.3 注册自定义算子

API	reg_custom_op
描述	注册自定义算子，仅支持 ONNX 模型。建议在 `config` 之后、`load_onnx` 之前调用，且必须在 `build` 之前调用。
参数	**custom_op**：自定义算子类，需实现 `op_type`、`shape_infer` 和 `compute`。`op_type` 建议以 'cst' 开头。
	custom_op_plugins：自定义算子插件共享库（.so）的路径。 若在此指定，`init_runtime` 无需再传；若不在此指定，则必须在 `init_runtime(custom_op_plugins=...)` 中传入，否则无法连板运行或做精度分析。 默认值为None。
返回值	0：注册成功。
	-1：注册失败。

自定义算子类示例如下：

```
import numpy as np
from rknn.api.custom_op import get_node_attr

class cstSoftmax:
    op_type = 'cstSoftmax'

    def shape_infer(self, node, in_shapes, in_dtypes):
        out_shapes = in_shapes.copy()
        out_dtypes = in_dtypes.copy()
        return out_shapes, out_dtypes

    def compute(self, node, inputs):
        x = inputs[0]
        axis = get_node_attr(node, 'axis')
        x_max = np.max(x, axis=axis, keepdims=True)
        tmp = np.exp(x - x_max)
        s = np.sum(tmp, axis=axis, keepdims=True)
        outputs = [tmp / s]
        return outputs
```

`custom_op_plugins` 两种传法的推荐用法：

```
# 方式一：在 reg_custom_op 中传入插件路径，init_runtime 自动获取
rknn.reg_custom_op(cstSoftmax, custom_op_plugins='/path/to/libcustom_op.so')
rknn.init_runtime(target='rk1820')

# 方式二：在 init_runtime 中传入插件路径
rknn.reg_custom_op(cstSoftmax)                          # 此处不传插件路径
rknn.init_runtime(target='rk1820',
                  custom_op_plugins='/path/to/libcustom_op.so')
```

7.4 模型推理

在进行模型推理前，必须先构建或加载一个RKNN模型。

> **注意**：独立的 `accuracy_analysis` 接口已不再维护，建议使用 `inference(accuracy_analysis=True)` 进行精度分析。

API	inference
描述	对当前模型进行推理，并返回推理结果。
参数	**inputs**：待推理的输入列表，格式为ndarray。
	data_format：输入数据的layout列表，“nchw”或“nhwc”，只对4维的输入有效。默认值为None，表示所有输入的layout都为NHWC。
	accuracy_analysis：是否开启逐层精度分析。 默认值为False。当 init_runtime 中设置了 target 参数时，进行板端（硬件）精度分析；板端精度分析时需在 config 中配置 profile_mode=True
	analysis_core_mask：指定核心进行精度分析。目前支持 0x1 到 0xff 之间的整数。默认值为 0，表示在所有核心上进行精度分析。
	output_dir：精度分析文件的输出目录，所有快照都保存在该目录。默认值为'./snapshot'。 如果init_runtime中没有设置target，在output_dir下会输出： - simulator目录：保存整个量化模型在simulator上完整运行时每一层的结果（已转成float32）； - golden目录：保存整个浮点模型在simulator上完整跑下来时每一层的结果; - error_analysis.txt：记录simulator上量化模型逐层运行时每一层的结果与golden浮点模型逐层运行时每一层的结果的余弦距离（entire_error cosine），以及量化模型取上一层的浮点结果作为输入时，输出与浮点模型的余弦距离（single_error cosine），更详细的信息请查看error_analysis.txt文件。 如果init_runtime中有设置target，则在output_dir里还会多输出： - runtime目录：保存整个量化模型在NPU上完整运行时每一层的结果（已转成float32）。 - error_analysis.txt：在上述记录的内容的基础上，还会记录量化模型在simulator上逐层运行时每一层的结果与NPU上逐层运行时每一层的结果的余弦距离（entire_error cosine）等信息，更详细的信息请查看error_analysis.txt文件。
返回值	results：推理结果，类型是ndarray list。

举例如下（完整代码参考examples/onnx/yolov5）：

```
# 使用模型对图片进行物体检测，得到物体类型和xy坐标
outputs = rknn.inference(inputs=[img], data_format=['nhwc'])
...
```

输出结果如下：

```
   class        score      xmin, ymin, xmax, ymax
-------------------------------------------------
   person       0.881     [ 209,  244,  286,  506]
   person       0.864     [ 478,  238,  559,  526]
   person       0.829     [ 110,  238,  230,  534]
   person       0.323     [  79,  353,  122,  517]
    bus         0.705     [  92,  131,  555,  465]
```

8 精度、性能和内存评估接口

8.1 精度分析

精度分析通过 7.4 模型推理 中的 `inference(accuracy_analysis=True)` 开启；板端精度分析时，需在 `config` 中配置 `profile_mode=True`。

8.2 评估模型性能

API	eval_perf
描述	获取模型性能。 模型必须运行在板端硬件上（与 PC 连接的指定设备，非模拟器）。在 `config` 中需配置 `profile_mode=True`
参数	**log_level**：打印性能信息等级，默认值为0。0：仅打印汇总表；1：打印每个 Op 的详细信息表和汇总表；2：打印每个 Op 的详细信息包括时间、周期和带宽及汇总表
	dynamic_idx：动态输入的索引值，仅在动态shape中有效。默认值为0。
返回值	0：运行成功。
	-1：运行失败。

举例如下：

```
# 对模型性能进行评估
ret = rknn.eval_perf()
```

8.3 获取内存使用情况

API	eval_memory
描述	获取模型在硬件平台运行时的内存使用情况。 模型必须运行在板端硬件上（与 PC 连接的指定设备，非模拟器）。
参数	无
返回值	0：运行成功。
	-1：运行失败。

举例如下：

```
# 对模型内存使用情况进行评估
ret = rknn.eval_memory()
```

8.4 获取设备列表

API	list_devices
描述	列出当前已连接的所有 ADB 设备。
参数	无。

API	list_devices
返回值	返回 adb_devices 列表（使用协处理器时列出的是主控设备编号列表），如果设备为空，则返回空列表。

举例如下：

```
adb_devices = rknn.list_devices()
```

返回的设备列表信息如下：

```
************************
all device(s) with adb mode:
adb_devices = 0123456789ABCDEF, 196.1.1.102:1820
************************
```

9 LLM和VLM转换接口

9.1 LoRA加载、开启和关闭

9.1.1 LoRA权重加载

API	**load_lora**
描述	加载LoRA权重。在加载模型(load_onnx/ load_llm)之后、在构建模型(build)之前加载LoRA权重
参数	**lora_paths**：LoRA权重文件（.safetensors后缀）嵌套路径列表。外层列表表示互斥的LoRA组；同一内层列表中的多个权重共用一套计算图。
	lora_config_paths：LoRA配置文件（.json后缀）嵌套路径列表，与 **lora_paths** 中对应内层列表的元素一一对应。
	lora_patterns：LoRA权重的模板名称嵌套列表，与 **lora_paths** 中对应内层列表的元素一一对应，作为关键字在推理阶段启动/关闭对应的LoRA。同一LoRA组中只能开启一个pattern。
	lora_quantized_dtype：LoRA权重的数据类型，目前支持的数据类型有int4、int6、int8、float16、bfloat16。默认为None；未指定时，当 **float_dtype** 为float16或bfloat16时跟随该配置，否则使用float16。
	lora_quantized_method：LoRA权重的量化方法，目前支持layer、 channel、group{SIZE}，默认为None。
	lora_node_map：可选的LoRA节点精确映射，支持传入Python字典或JSON文件路径。传入后使用映射中的基础节点名和A/B权重key，不再使用前缀/后缀匹配；不能同时设置 **lora_prefix**、**lora_postfix_a** 或 **lora_postfix_b**。
	lora_prefix：LoRA权重 key 的前缀，默认为 'base_model.model.'。
	lora_postfix_a：LoRA A 权重 key 的后缀，默认为 '.lora_A.weight'。
	lora_postfix_b：LoRA B 权重 key 的后缀，默认为 '.lora_B.weight'。
	lora_distribute_strategy：LoRA权重的多核拆分方式, 目前支持 'best_perf', 'less_mem'. 默认为'best_perf'
返回值	0：加载成功。
	-1：加载失败。

> **注：** 未传入 **lora_node_map** 时，LoRA key 按 `prefix + 转换后的模型节点名 + postfix A/B` 匹配。模型节点名会依次将 `/` 替换为 `.`、删除末尾的 `MatMul`，再去除首尾的 `.`。

举例如下：

```
# 从当前目录加载LoRA权重
rknn.load_lora(
    lora_paths=[
        ["./slot0_lora0.safetensors", "./slot0_lora1.safetensors"],
        ["./slot1_lora0.safetensors", "./slot1_lora1.safetensors"],
    ],
    lora_config_paths=[
        ["./slot0_lora0.json", "./slot0_lora1.json"],
        ["./slot1_lora0.json", "./slot1_lora1.json"],
    ],
    lora_patterns=[
        ["slot0_lora0", "slot0_lora1"],
        ["slot1_lora0", "slot1_lora1"],
    ],
    lora_distribute_strategy='best_perf',
)
```

使用自定义节点映射时，`lora_node_map` 的结构示例如下：

```
lora_node_map = {
    "decoder.layers.0.self_attn.q_proj": [
        [
            ["adapter.g0.p0.lora_A.weight", "adapter.g0.p0.lora_B.weight", 0],
            ["adapter.g0.p1.lora_A.weight", "adapter.g0.p1.lora_B.weight", 1],
        ],
        [
            ["adapter.g1.p0.lora_A.weight", "adapter.g1.p0.lora_B.weight", 0],
            ["adapter.g1.p1.lora_A.weight", "adapter.g1.p1.lora_B.weight", 1],
        ],
    ],
}
rknn.load_lora(
    lora_paths=lora_paths,
    lora_config_paths=lora_config_paths,
    lora_patterns=lora_patterns,
    lora_node_map=lora_node_map,
)
```

顶层 key 是加载后 IR 中的基础节点名；value 的第一层列表对应 LoRA group；每条记录为 `[a_key, b_key, weight_index]`，其中 `a_key`、`b_key` 必须是对应 safetensors 文件中的真实 key，`weight_index` 是该 group 内的文件索引。也可以将相同结构保存为 JSON 文件，并传入 `lora_node_map='./lora_node_map.json'`。

9.1.2 开启LoRA

API	**enable_lora**
描述	开启指定LoRA。
参数	**enable_patterns**：开启LoRA的模板名称列表。
	scales：开启LoRA的缩放系数列表，例如 [1.0, 0.8]。默认为None，表示从LoRA配置文件中读取缩放系数。
返回值	0：开启成功。

API	enable_lora
	-1：开启失败。

举例如下：

```
# 开启指定LoRA
rknn.enable_lora(enable_patterns=["slot0_lora0"])
```

9.1.3 关闭LoRA

API	disable_lora
描述	关闭指定LoRA。
参数	**disable_patterns**：关闭LoRA的模板名称列表。
返回值	0：关闭成功。
	-1：关闭失败。

举例如下：

```
# 关闭LoRA
rknn.disable_lora(disable_patterns=["slot0_lora0"])
```

9.2 查询功能

API	query
描述	查询模型相关信息或状态。
参数	**cmd**：查询命令的类型，格式为字符串。
返回值	results：查询的结果。

9.2.1 查询命令的类型

命令	描述
QUERY_ROPE_CACHE	查询模型rope输入的sin/cos的cache。

举例如下：

```
# 查询模型rope输入的sin/cos的cache
rope_cache = rknn.query('QUERY_ROPE_CACHE')
```

9.3 KVCache控制器

KVCache控制器按各个 attention 配置独立维护缓存状态。除手动生成控制输入外，也可以把模型中已识别的 RoPE/cache control 输入设为 `None`，由 `inference()` 自动补齐；绑定和 mask 有效值配置见 9.3.4。

9.3.1 获取控制kvcache的输入

API	kvcache_controller.generate_kvcache_control_tensors
描述	获取控制kvcache的输入和模型动态shape的id。
参数	**input_seq_len**： 模型输入token长度。
参数	**system_prompt_len**：系统提示词的token长度，默认值为0。
参数	**history_seq_len**：指定本次推理使用的历史长度；分页模式下小于缓存位置 `Ti` 时触发回退，非零回退位置须同时按 `kvcache_group_size` 和 32 对齐。默认沿用控制器状态。
参数	**request_mask**：分页模式下每个请求的启用标志，值为 0 或 1；可以传标量或长度为 `request_num` 的序列，默认全为 1。
返回值	返回 `(attention_inputs, dynamic_idx)`。`attention_inputs` 外层按输入分块排列，每个分块按 attention 配置顺序展开各配置的控制张量；单 attention 的返回格式保持兼容。`dynamic_idx` 为对应模型动态 shape 的 id。

分页模式下，每轮每个 attention 配置返回一个 `int32` 控制张量，形状为 `[request_num, 9]`，列顺序为 `Th, Tc, Ts, Tsr, longest_surviving_id, store_begin_id, Tr, Ti, request_mask`。不同 attention 的容量和缓存指针分别计算，不能在 SlidingAttention 和 FullAttention 之间复用一份控制张量。单请求可以分块预填充；多请求仅支持每请求 `Tc` 为 0 或 1 的解码。`request_mask=0` 或 `Tc=0` 的请求不会推进缓存状态。

手动生成接口会推进控制器记录的状态，调用者应依次执行返回的分块，并自行处理推理失败。也可以继续完全手动提供 RoPE 和控制输入，不使用自动补齐。

分页模式下，系统提示生效后须保持 `Ts/Tsr` 不变；非零回退目标不能小于保留系统提示区长度 `Ts`。需要更换系统提示时，应先清空状态并重新处理输入。

举例如下：

```
attention_inputs, dynamic_idx =
rknn.kvcache_controller.generate_kvcache_control_tensors(input_seq_len)
```

9.3.2 清空kvcache状态

API	kvcache_controller.clear_kvcache_status
描述	清空所有 attention 配置的控制器状态，下次推理从空历史开始；不释放已分配的缓存页。
参数	无。
返回值	无。

举例如下：

```
rknn.kvcache_controller.clear_kvcache_status()
```

9.3.3 回退缓存位置

API	kvcache_controller.roll_back(roll_back_len)
描述	从当前控制器历史长度中减去指定 token 数，使下一轮各 attention 分别使用回退后的位置。回退量不能超过已有历史，也不能回到已被环形缓存淘汰的位置。分页模式下，非零目标位置须同时按 `kvcache_group_size` 和 32 对齐；接口立即检查全部配置，请求不合法时保留原状态。

API	kvcache_controller.roll_back(roll_back_len)
参数	**roll_back_len**：非负整数或长度为 `request_num` 的非负整数序列。
返回值	无。

```
rknn.kvcache_controller.roll_back(32)
```

9.3.4 绑定并自动补齐控制输入

API	kvcache_controller.bind_kvcache(mask_valid_value=1, mask_invalid_value=0)
描述	板端绑定当前模型已配置的 KV 缓存页，并配置当前 token mask 的有效值和 padding 值；推理时按模型元数据自动生成缺失的 RoPE/control 输入。须在加载模型并初始化 runtime 后调用；本接口不提供缓存切换。
参数	**mask_valid_value**：mask 中有效 token 的值，默认 1；可传标量，或以各配置的 `mask_name` 为键的字典。
参数	**mask_invalid_value**：mask 中 padding 的值，默认 0；须与有效值不同，支持标量或同样的字典。
返回值	成功返回 0；不合法的配置抛出异常。

`inference()` 会按各 attention 配置的 `mask_name` 读取当前输入 mask，从中推导有效 token 数 `Tc`；历史长度 `Th` 由控制器维护，mask 不包含历史。自动模式仅支持 `(B, N)` mask，且每行有效 token 在前、padding 在后；`(B, N, 1, N)` 等高维 mask、左 padding、有效区域有空洞或存在未配置的 mask 值时会报错。单请求须至少有一个有效 token；分页多请求仅支持每行有效 token 数为 0 或 1 的解码。这里的 mask 有效值配置不会改变分页控制张量内 `request_mask` 的 `0=停用、1=启用` 约定。

将模型元数据 `rope_cache_names` 和 `control_input_names` 标识的输入位置设为 `None` 即可自动生成对应张量。其他模型输入仍须显式提供；输入顺序以转换后模型为准。未显式绑定时，首次自动推理采用默认的 `1=有效、0=padding` 配置。补齐只作用于本次调用的输入副本，调用者的 `None` 占位符可以重复使用。

同一轮所有 attention 的控制输入须全部为 `None` 或全部显式提供。手动生成控制张量后，应把生成结果作为该轮的控制输入；不要再用 `None` 重复生成。RoPE 可以单独设为 `None`，此时按显式控制张量中的位置和长度补齐，不会再次推进历史。自动控制在 KV 写入成功后提交状态；写入结果不确定的失败会使状态失效，须清空控制器状态后重新开始。

重新初始化或更换 runtime 时，若已有非零缓存历史，须先调用 `clear_kvcache_status()` 再重新绑定。控制器保留已配置的 mask 有效值和 padding 值，旧 runtime 的缓存历史不能带入新 runtime。

`SKIP_WRITE` 不推进缓存状态。自动控制支持 `DEFERRED_WRITE` 的合并调用（模型 IR 的 `infer_mask=0`）；非零 `infer_mask` 的分阶段调用须显式提供控制输入。

自动补齐暂不支持 `accuracy_analysis=True`；精度分析时须显式提供全部 RoPE/control 张量，沿用手动输入方式。

普通文本 RoPE 在 `position_embeddings_host_storage=True` 时使用 `QUERY_ROPE_CACHE` 查询的缓存表，按当前逻辑历史和有效长度选择位置，并补齐到本次输入 shape；为 `False` 时提供完整 RoPE 表，由算子根据历史和有效长度选取。MRoPE 等需要额外位置输入的路径仍须由调用者提供位置参数。

单 attention、分页模式的输入顺序为 embedding、mask、logits 索引、两份 RoPE 表和控制张量时，用法如下：

```
ret = rknn.kvcache_controller.bind_kvcache(mask_valid_value=1,
mask_invalid_value=0)
if ret != 0:
    raise RuntimeError('Bind KVCache controller failed')
inputs = [input_embeds, attention_mask, num_logits_to_keep, None, None, None]
outputs = rknn.inference(inputs, data_format=['nchw'] * len(inputs))
```

对于 `0=有效、1=padding` 的模型，绑定时交换两个参数即可。板端缓存页容量仍由 `alloc_kvcache_pages()` 和 `grow_kvcache_pages()` 管理，可以沿用预先为输入和生成预算分配容量的方式。

10 ONNX→PyTorch 模型生成与重建

ONNX→PyTorch 功能可将 RKNN build 生成的中间模型转换为可读、可编辑的 PyTorch `nn.Module`，用于 PyTorch Simulator、逐层精度分析，以及在修改模型代码、权重或量化 参数后重新生成 RKNN 模型。默认关闭该功能；`generate_torch_model=False` 时不改变原有 build/export 流程。

10.1 生成 PyTorch 模型

在 `RKNN.config` 中设置 `generate_torch_model=True`，正常执行模型加载、build 和 `export_rknn`：

```
from rknn.api import RKNN

rknn = RKNN(verbose=True)
rknn.config(
    target_platform="rk182x",
    generate_torch_model=True,
)
rknn.load_onnx(model="model.onnx")
rknn.build(do_quantization=True, dataset="dataset.txt")
rknn.export_rknn("model.rknn")
rknn.release()
```

非量化模型设置 `do_quantization=False`，且不需要 `dataset`。主要生成产物如下：

产物	说明
`<build_cache_dir>/torch_model/generated.py`	可编辑的 PyTorch 模型代码。
`<build_cache_dir>/torch_model/generated_weights.pth`	模型权重及量化参数 buffer。
`<build_cache_dir>/torch_model/model_generated.onnx`	`rebuild` 使用的回导 ONNX。
`<build_cache_dir>/torch_model/input_preprocess.json`	输入预处理配置。
`<build_cache_dir>/torch_model/meta.json`	`rebuild` 所需的模型和原始 config 信息。
`<model>_quant_tab.json`	量化模型的量化参数 manifest。

这些文件属于同一组产物。复制或归档时，应保留整个 `torch_model` 目录、 `quant_tab.json` 及其引用的 PTH 文件，不能混用不同 build 生成的文件。

对于通过 `load_lora` 加载的 LoRA 模型，生成产物会同时保留 LoRA 权重、pattern/group 及启用状态。生成的 PyTorch 模型可通过 `set_lora_state()` 切换 LoRA，`rebuild` 会继续 使用这些 LoRA 元数据。

10.2 使用 PyTorch Simulator

将生成产物目录传给 `init_runtime` 即可使用 PyTorch Simulator：

```python
from rknn.api import RKNN

rknn = RKNN(verbose=True)
rknn.config(target_platform="rk182x")
rknn.init_runtime(
    target="simulator",
    simulator_backend="pytorch",
    pytorch_artifact_dir="tmp/torch_model",
    pytorch_quant_json="model_quant_tab.json",
)
outputs = rknn.inference(inputs=[input_data])
rknn.release()
```

量化模型建议显式传入配套的 `pytorch_quant_json`；浮点模型不需要该参数。逐层精度分析 仍通过 `inference` 启用：

```python
outputs = rknn.inference(
    inputs=[input_data],
    accuracy_analysis=True,
    output_dir="./accuracy_analysis",
)
```

`simulator_backend="pytorch"` 只选择软件模拟器后端，不改变最终 RKNN 模型的板端执行方式。

10.3 加载和修改生成模型

如需将生成产物作为普通 PyTorch 模型运行，只使用公开加载接口：

API	说明
`load_generated_module(py_path)`	从明确的 `generated.py` 路径加载生成模块，返回包含 `Model` 类的 Python module。
`load_model_from_weight_store(model_class, base_path, map_location="cpu", strict=True, mmap=True)`	从 `base_path` 对应的权重存储加载 `model_class`，返回 `(model, state_dict)`。`base_path` 传文件前缀，例如 `tmp/torch_model/generated`，无需添加 `_weights.pth`。

```python
import torch

from rknn.api.onnx_torch_codegen import (
    load_generated_module,
    load_model_from_weight_store,
)

artifact_dir = "tmp/torch_model"
module = load_generated_module(f"{artifact_dir}/generated.py")
model, _ = load_model_from_weight_store(
    module.Model,
    f"{artifact_dir}/generated",
    map_location="cpu",
)
model.eval()

with torch.inference_mode():
    outputs = model.inference(torch.from_numpy(input_data))
```

不要直接导入 Toolkit 内部实现模块；这些路径不是公开 API。用户只需通过上述公开接口加载 产物。可以修改 `generated.py` 中的算子或 `forward`，也可以修改权重文件，但必须保持 输入、输出和 shape 契约有效。生成模型需要配套版本的 RKNN3-Toolkit，不能作为完全独立的 PyTorch 模型发布。

10.4 使用生成模型重建 RKNN

修改生成代码、权重或量化参数后，基于原 build cache 执行 `rebuild`：

```python
from rknn.api import RKNN

rknn = RKNN(verbose=True)
rknn.rebuild(
    build_cache_dir="tmp",
    quant_json="model_quant_tab.json",
    dataset="dataset.txt",
)
rknn.export_rknn("model_rebuilt.rknn")
rknn.release()
```

- 只替换兼容量化参数且图结构不变时，不需要重新校准，可以省略 `dataset`；
- 修改量化模型的图结构后，需要提供 `dataset`；
- 未指定 `quant_json` 时，使用 build cache 记录的原量化参数；
- `rebuild` 在临时 staging 目录中执行，成功后才更新原 cache；
- 最终部署效果应以 rebuild 后的 RKNN 模型和板端推理结果为准。

10.5 量化参数存储与替换

`generated.py` 对 forward QDQ 和重建所需的量化 tensor 只声明 buffer 的名称、shape 和 dtype，不保存 weight/activation scale、zero-point、min、max 或量化整数权重的实际数值； 实际数值统一保存在 `generated_weights.pth`，并由 `quant_tab.json` 建立 ONNX tensor 到 PTH buffer 的映射。这种设计使同 schema 的量化参数可以替换而无需编辑生成源码，同时 通过固定 shape/dtype 防止加载不兼容配置。输入 mean/std 或 fused-mean/std 等输入预处理 配置仍会记录在生成模型元数据中，它们不属于可通过 quant manifest 替换的图内 QDQ buffer。

`quant_tab.json` 的 tensor 条目包含 `dtype`、`qmethod`、`qtype` 以及各参数的 PTH buffer key；顶层 `_torch_weights` 指向 PTH 文件，并相对于 JSON 所在目录解析。少数模型还包含 `_torch_quant_params`，此时也必须保留其引用的 companion PTH。

若两个模型的图结构、tensor 条目、buffer key、参数 shape 和量化 schema 完全一致，可复制 原 manifest 并将 `_torch_weights` 改为新 PTH 路径，再把该 manifest 传给 `rebuild`：

```
rknn.rebuild(
    build_cache_dir="model_b/tmp",
    quant_json="custom_quant_tab.json",
)
```

不能把裸 PTH 直接作为 `quant_json` 传入。若修改量化类型、量化方法或 group size 导致 buffer shape/dtype 改变，应重新生成整套 Torch 产物，不能只替换 PTH。

10.6 示例与注意事项

量化参数替换示例位于 `examples/functions/rebuild_with_custom_qparam/`。该示例构建两组不同校准参数，并使用一组模型的 manifest/PTH 重建另一组同结构模型；可通过以下方式运行：

```
cd examples/functions/rebuild_with_custom_qparam
python3 test.py
python3 test.py --device-id <device-id>
```

使用时还需注意：不要混用不同 build 的 `generated.py`、PTH、manifest 和 `meta.json`； 修改 scale 或 zero-point 时，应同步保证 min/max 与量化整数权重一致；修改输入、输出或图 结构后，应重新提供量化 dataset。

11 RKQuantizer

RKQuantizer 是 RKNN3-Toolkit 提供的独立量化组件，用于在 PyTorch 浮点模型导出为 ONNX 之前，先对其中的 Linear 层权重进行训练后量化（Post-Training Quantization）。RKQuantizer 直接作用于 `torch.nn.Module`，量化完成后返回一个 fake-quant 模型，再由用户将其导出为 ONNX，最后通过 `rknn.load_llm` 接口载入并调用 `rknn.build` 完成 RKNN 模型转换。典型流程如下：

```
浮点 PyTorch 模型 ──RKQuantizer──▶ fake-quant PyTorch 模型 ──导出 ONNX──▶
rknn.load_llm ──▶ rknn.build
```

使用前需从 `rknn.quantization.api` 导入 RKQuantizer：

```
from rknn.quantization.api import RKQuantizer
```

RKQuantizer 的使用分为接口调用、校准数据准备、AWQ 量化配置、混合精度量化与 GRQ 显存占用五部分，分别见 11.1、11.2、11.3、11.4 与 11.5 小节。

支持的模型限制

RKQuantizer 并非支持任意 PyTorch 模型，使用时需满足以下前提：

1. **模型架构**：目前主要支持 Transformer 架构的模型（如各类 LLM、VLM 的语言/视觉 部分）。
2. **量化对象**：RKQuantizer 仅对模型中的 `torch.nn.Linear` 层进行量化，其它类型的算子（如卷积、Embedding、LayerNorm 等）不参与量化，保持原精度。
3. **GRQ 作用范围**：GRQ 量化算法（`quantized_algorithm='grq'`）会自动检索传入模型中的 `torch.nn.ModuleList`（找不到时回退检索 `torch.nn.Sequential`）作为 transformer block 序列；若模型中两者均不存在，会报 "No nn.ModuleList or nn.Sequential found in the passed model, the grq quantization algorithm can not be used" 错误。因此模型需以 `ModuleList` 或 `Sequential` 形式组织其重复堆叠的 block。注意，如果模型里面有多个ModuleList，那么会以长度最长的ModuleList为目标执行量化。AWQ 的作用范围也以此 block 序列为准。
4. **设备要求**：GRQ 算法建议在 CUDA 或 MPS 环境下运行（CPU 下耗时会很长）。Normal 算法可以在 CUDA、MPS 或 CPU 环境下运行。

11.1 接口说明

以下为 RKQuantizer 各接口的详细说明。

API	**RKQuantizer**
描述	初始化 RKQuantizer 量化工具对象。
参数	**verbose**：是否打印详细日志信息，可取 True、False 或 'debug'。默认值为 False。
	verbose_file：日志文件路径。设置该参数且 verbose 为 True 时，日志信息会同时写入该文件；若其所在目录不存在则该参数被忽略。默认值为 None。
返回值	RKQuantizer 对象。
API	**load_model**
描述	将待量化的 PyTorch 浮点模型及对应的 tokenizer 等加载进量化工具。

API	load_model
参数	**model**：待量化的 PyTorch 模型（`torch.nn.Module`）。该参数指定了实际参与量化的子模型，非常关键。RKQuantizer 仅对传入的 `model` 进行量化。当使用**grq量化算法**时，**传入的 model 内应最好只包含一个 `nn.ModuleList` / `nn.Sequential` 主干容器（主干容器内部的 `nn.ModuleList` / `nn.Sequential` 无影响)**：若检测到多个，RKQuantizer 仅告警并默认选取元素个数最多（`len` 最大）的一个作为量化目标继续执行（这通常意味着传入了过大的子模型，例如把整个 VLM 传进来，其中 vision 与 llm 各有一组 block 列表）；请按告警提示改传正确的待量化子模块（如 `model.model` / `model.visual`）。通常取值分两种情况： **- 纯 LLM 模型**：直接传入加载得到的 CausalLM 模型本身（`model=model`）。 **- VLM（多模态）模型**：VLM 由 vision 部分与 llm 部分组成，二者需分别量化，**不可传入整个 VLM 模型**，而应按量化目标传入其对应子模块： · 量化 vision 部分时，传入 vision 子模块（如 `model.visual`、`model.model.vision_model`、`model.model.vision_tower` 等，视模型结构而定）； · 量化 llm 部分时，传入语言模型子模块（如 `model.model`、`model.model.text_model`、`model.model.language_model`、`model.language_model` 等，视模型结构而定）。各模型对应子模块的准确名称可通过打印模型结构确认。
	model_lora：LoRA 权重路径（HuggingFace 格式的 LoRA adapter 目录）， 默认值为 None。设置该参数后，RKQuantizer 会读取该目录下的 `adapter_model.safetensors`（LoRA 权重）与 `adapter_config.json`（LoRA 配置），将其中 `lora_A` / `lora_B` 权重统一转换为与模型一致的 dtype，并在量化阶段按 Linear 层名称后缀（`<layer_name>.lora_A.weight`、`<layer_name>.lora_B.weight`）匹配挂载到对应 Linear 层上，缩放系数 `scale = lora_alpha / rank`（`lora_alpha` 取自 `adapter_config.json`，`rank` 为 `lora_B` 的输出维度）。键名匹配兼容带 `base_model.model.` 前缀（PEFT 默认导出格式）或不带前缀的情况，但要求同一后缀在 LoRA 权重中唯一匹配，否则会报错。 **注意**：(1) 该功能主要配合 `quantized_algorithm='grq'` 使用，使 GRQ 感知 LoRA 适配；(2) 当传入 `model_lora` 时，AWQ 阶段会被自动跳过（即 AWQ 与 LoRA 不可同时使用），即使设置了 `awq_config` 也不会执行 AWQ。
	tokenizer：与模型配套的 tokenizer，用于在校准数据为"纯文本 JSON 模式"时对文本进行编码，校准数据为"hook + pickle 模式"时，可以设置为None。
	device：量化执行设备，当前支持 'cuda'、'mps'、'cpu' 或者 'auto'， 默认值为 'cuda'。设置为 'auto' 时会按 PyTorch 后端可用性自动选择 'cuda' > 'mps' > 'cpu'，而不是按系统名称判断。执行 'grq' 量化算法时，建议使用 'cuda' 或可用的 'mps'；如果最终选择 CPU，会给出慢速警告。
	system_prompt：系统提示词（字符串）。仅对"纯文本 JSON 模式"校准数据生效。当 `system_prompt` 与 `tools` 任一不为 None 时，每条样本的 `input+target` 文本会经 tokenizer 的 chat template 拼装成对话，使校准输入分布更贴近真实部署场景；二者均为 None 时样本文本原样编码，不套用对话模板。其中 `system_prompt` 不为 None 时会在对话中插入一条系统消息，否则只构造用户消息。 默认值为 None。"hook + pickle 模式"校准数据下该参数不生效。
	tools：工具定义列表，透传给 `tokenizer.apply_chat_template` 的 `tools` 参数。`tools` 或 `system_prompt` 任一不为 None 即触发对话模板套用，用于构造带工具调用（function calling）的提示；可单独传入 `tools`（不带 `system_prompt`）以匹配无系统提示词但启用 function calling 的部署场景。 默认值为 None。
	system_role：套用对话模板时系统消息所使用的 role 名称（仅在 `system_prompt` 不为 None 时使用），默认值为 'system'。
	user_role：套用对话模板时用户消息所使用的 role 名称，默认值为 'user'。

API	**load_model**
返回值	0：加载成功。 -1：加载失败。
API	**quantize**
描述	对已加载的 PyTorch 模型执行量化，返回 fake-quant 后的 PyTorch 模型。**注意：该接口返回的是量化后的模型对象，而非 0/-1 状态码。**
参数	**quantized_dtype**：量化类型。当前支持 w4a16, w6a16, w8a16，默认值为 w4a16(建议使用)。其中 w4a16 表示权重为 4 bit 非对称量化、激活值为 16 bit 浮点精度。
	quantized_method：权重量化方式。当前仅支持'group32'， 默认值为 'group32'。'group32' 表示在 per-channel量化 基础上，每个输出通道在输入通道方向上按 32 细分为多个小组，每组一套量化参数，通常精度更高。
	quantized_algorithm：量化算法。当前支持 'grq' 和 'normal'， 默认值为 'grq'。GRQ 会在量化基础上对模型权重进行微调，精度更高，建议在 CUDA 或 MPS 环境下运行，且需要给定量化校准数据集；CPU 可运行但速度很慢，会给出警告。normal 量化无需量化校准数据集。
	dataset：校准数据集路径，用于量化参数统计与 GRQ 微调。一般指向一个 json 文件（其中包含若干条校准样本），校准数据集有严格的规则，具体说明见11.2章节。如果设置dataset为None，那么会 quantized_algorithm 会被强制替换为 normal 量化算法
	awq_config：AWQ 量化配置，字典类型。配置后会先执行 AWQ 再执行 GRQ；为 None 时跳过 AWQ。字典需描述模型 transformer block 的结构，包含 BLOCKNAME（block 模块名，如 'Qwen3DecoderLayer'）、 ATTN_NORM/ATTN_Q/ATTN_K/ATTN_V/ATTN_QKV/ATTN_KV/ATTN_OUT、 FFN_NORM/FFN_UP/FFN_GATE/FFN_DOWN 等子模块名。 默认值为 None。 此外，当 `load_model` 传入了 LoRA 权重（`model_lora` 参数）时，AWQ 会被自动跳过（AWQ 与 LoRA 不可同时使用）。AWQ 量化配置具体说明见11.3章节
	auto_hybrid_rate：自动混合精度比例，取值为 [0, 1) 的浮点数， 默认值为 0（关闭）。 开启后会按该比例自动挑选部分敏感 Linear 层使用更高精度的 w8a16 量化，以提升精度。混合量化具体说明见11.4章节。**注：该参数不可与 layer_quant_config 同时使用。**
	layer_quant_config：按层指定量化类型的配置，支持传入字典或 JSON 文件路径。格式为 `{layer_pattern: "w8a16"}`，其中 layer_pattern 为用于匹配 Linear 层名称的 shell-style 通配模式（例如 `model.layers.*.mlp.down_proj` ），匹配到的层将使用指定类型量化。当前支持 "w6a16" 或者 "w8a16" 取值。该参数不可与 auto_hybrid_rate 同时使用。 默认值为 None。 混合量化具体说明见11.4章节。
返回值	伪量化后的 `torch.nn.Module` 模型，对应load_model函数中model参数的传入模型。失败时通过 RKNNLog 抛出错误。
API	**export_op_quantized_dtype**
描述	当开启了混合精度量化（layer_quant_config 或 auto_hybrid_rate）时，将各层实际使用的量化类型导出为 JSON 文件，便于在后续 `rknn.config` 中通过 `op_quantized_dtype` 参数对齐配置。未开启混合精度时该接口无需调用，调用会给出告警并直接返回。
参数	**onnx_path**：导出后的 ONNX 模型路径，用于建立 torch 层名与 ONNX 算子之间的映射。
	op_dtype_path：导出的量化类型 JSON 文件路径，默认值为 'layer_bit.json'。
返回值	无。

举例如下：

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from rknn.quantization.api import RKQuantizer

# 1. 加载浮点模型与 tokenizer
model_path = 'Qwen/Qwen3-1.7B'
model = AutoModelForCausalLM.from_pretrained(model_path, torch_dtype=torch.float16,
trust_remote_code=True)
tokenizer = AutoTokenizer.from_pretrained(model_path, trust_remote_code=True)

# 2. 初始化量化工具并加载模型
QuantTool = RKQuantizer(verbose=True)
ret = QuantTool.load_model(model=model, tokenizer=tokenizer, device='cuda')
if ret != 0:
    print('Load model failed!')
    exit(ret)

# 2.1（可选）若部署时模型带有系统提示词/工具调用，可让校准输入套用同样的对话模板，
#     使校准分布与真实部署一致（仅对"纯文本 JSON 模式"校准数据生效）：
# system_prompt = "You are a helpful assistant."
# tools = [...]  # 工具定义列表，透传给 tokenizer.apply_chat_template
# ret = QuantTool.load_model(model=model, tokenizer=tokenizer, device='cuda',
#                            system_prompt=system_prompt, tools=tools,
#                            system_role='system', user_role='user')

# 3. 执行量化（GRQ + w4a16 + group32），返回 fake-quant 后的模型
model = QuantTool.quantize(quantized_dtype="w4a16",
                           quantized_method="group32",
                           quantized_algorithm='grq',
                           dataset='./dataset.json')
model = model.cpu()

# 4. 将量化后的模型导出为 ONNX（导出时 attention 必须切回 eager 实现）
#    export_to_onnx(model, './Qwen3-1.7B.onnx', ...)
```

若部署模型带有 LoRA 适配（HuggingFace 格式，例如通过 PEFT 训练得到的 adapter），可在 `load_model` 时通过 `model_lora` 传入 LoRA 权重目录，使 GRQ 在「基座模型 + LoRA」的等效权重上执行量化微调。传入的 `model` 应为未合并 LoRA 的基座模型本身，且此时 AWQ 会被自动跳过：

```python
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from rknn.quantization.api import RKQuantizer

# 基座模型与 LoRA adapter（HuggingFace 格式；目录下需含 adapter_model.safetensors 与
adapter_config.json）
model_path = 'Qwen/Qwen3-1.7B'
lora_path = './qwen3_lora_adapter'

model = AutoModelForCausalLM.from_pretrained(model_path, torch_dtype=torch.float16,
trust_remote_code=True)
tokenizer = AutoTokenizer.from_pretrained(model_path, trust_remote_code=True)

QuantTool = RKQuantizer(verbose=True)
# 通过 model_lora 传入 LoRA 权重目录，GRQ 将在基座 + LoRA 的等效权重上量化
ret = QuantTool.load_model(model=model, model_lora=lora_path, tokenizer=tokenizer,
device='cuda')
if ret != 0:
    print('Load model failed!')
    exit(ret)

# 传入 LoRA 时 AWQ 会被自动跳过（AWQ 与 LoRA 不可同时使用），仅执行 GRQ
model = QuantTool.quantize(quantized_dtype="w4a16",
                           quantized_method="group32",
                           quantized_algorithm='grq',
                           dataset='./dataset.json')
model = model.cpu()
```

对于 VLM（多模态）模型，需分别对其 vision 部分与 llm 部分执行量化，每次调用 `load_model` 时只传入对应子模块，而非整个 VLM。以 Qwen2.5-VL 为例：

```python
from transformers import Qwen2_5_VLForConditionalGeneration, AutoProcessor

model_path = 'Qwen/Qwen2.5-VL-3B-Instruct'
vlm = Qwen2_5_VLForConditionalGeneration.from_pretrained(model_path,
torch_dtype=torch.float16, trust_remote_code=True)

QuantTool = RKQuantizer(verbose=True)

# 量化 vision 部分：传入 vision 子模块，tokenizer 可根据校准数据情况设为 None
ret = QuantTool.load_model(model=vlm.visual, tokenizer=None, device='cuda')
# ... 调用 quantize 导出 vision 的 ONNX ...

# 量化 llm 部分：传入语言模型子模块，tokenizer 可根据校准数据情况设为 None
ret = QuantTool.load_model(model=vlm.model, tokenizer=None, device='cuda')
# ... 调用 quantize 导出 llm 的 ONNX ...
```

不同 VLM 的子模块命名略有差异，常见对应关系如下（具体以打印模型实际结构组件为准）：

量化目标	常见子模块路径示例
vision 部分	`model.visual`、`model.model.vision_model`、`model.model.vision_tower`、`model.vision_model`
llm 部分	`model.model`、`model.model.text_model`、`model.model.language_model`、`model.language_model`

11.2 校准数据集准备

`quantize` 接口在执行 GRQ 量化（以及 AWQ、自动混合精度）时，需要使用校准数据集（`dataset` 参数）来统计各层激活分布并对权重进行微调。校准数据集以一个 **JSON 文件** 的形式提供，文件内容为一个列表，列表中每个元素为一条样本。RKQuantizer 会逐条读取样本并构造模型输入，目前支持以下两种样本格式，用户可根据模型类型自行选择。

方式一：hook + pickle（通用方式，推荐）

最通用的方式是直接获取**量化模型**（即传入 `load_model` 的那个子模块，如纯 LLM 的 `model`、VLM 的 `model.model` / `model.visual`）的真实输入，通过 PyTorch 的 hook 机制在浮点模型前向时捕获该子模块的输入，用 pickle 序列化保存，再构建 JSON 索引文件。该方式适用于所有模型，包括多模态模型（VLM）、语音模型等非纯文本输入的模型。

> **注意**：hook 捕获的是**量化模型整体（即 `load_model` 传入的子模块）的输入**，而不是该模型内部第一个 transformer block 的输入。RKQuantizer 在量化时会将这份输入整体喂给量化模型，再在其内部逐层统计激活，因此捕获点必须在量化模型的最外层入口上。

JSON 文件中每条样本为一个字典，仅包含一个 `sample` 字段，其值为 pickle 文件的相对路径（相对于该 JSON 文件所在目录）。pickle 文件内容为一个字典，包含 `args`（位置参数）与 `kwargs`（关键字参数）两个键，分别对应量化模型前向时的位置输入与关键字输入。格式如下：

```
[
  {"sample": "model_inputs/sample_0"},
  {"sample": "model_inputs/sample_1"}
]
```

获取该数据的标准做法是：在待量化的浮点模型上，对量化模型（即 `load_model` 传入的子模块）注册 `register_forward_pre_hook`（需开启 `with_kwargs=True` 以同时捕获关键字参数），在 hook 中将输入存入列表并抛出异常以提前终止前向（避免执行完整推理），随后将捕获的输入 pickle 落盘。为避免各 demo 重复编写 hook 样板代码，rknn3-model-zoo 将该机制封装为公共工具 `py_utils/cali_data.py`，提供 `capture_module_input(hook_module, run, *args, **kwargs)` 函数：它把"注册 forward pre hook → 调用模型 → 捕获 `(args, kwargs)` → 抛 `StopForward` 短路前向 → 移除 hook"打包成一行，调用方无需感知 hook 细节。其内部 hook 会 `raise StopForward` 终止被捕获子模块的前向，因此即便传入 `model.generate` 也只会跑一次前向即中断，不会真正生成 token。举例如下：

```python
import os
import json
import pickle
import torch
from transformers import AutoProcessor
# 引入公共 hook 捕获工具（路径视工程结构调整）
from py_utils.cali_data import capture_module_input

model_path = 'Qwen/Qwen2.5-VL-3B-Instruct'
model = ...                          # 加载浮点模型，torch_dtype 与量化时保持一致
processor = AutoProcessor.from_pretrained(model_path)

# 逐条样本前向，一行捕获进入待量化子模块的输入
info = []
datasets = json.load(open('./dataset.json', 'r'))    # 原始标注数据（含图像/问题等）
for idx, data in enumerate(datasets):
    inputs = processor(...)                  # 按模型要求构造完整输入
    inputs = inputs.to(model.device)
    with torch.inference_mode():
        # hook_module 传量化模型本身（即 load_model 传入的子模块，这里量化 vision 部分即传
model.visual），
        # 捕获的是该子模块最外层入口的输入，而非其内部第一个 transformer block 的输入。
        # run 传要触发的模型调用（lambda 包裹，可含 prepare_inputs_embeds 等前置步骤）
        temp = capture_module_input(
            model.visual,
            lambda: model(**inputs)
        )
        # temp 即 {"args": (...), "kwargs": {...}}；若本次输入未流经该子模块则为 None

    # 将捕获的输入 pickle 落盘，并记录到索引 JSON
    sample_name = 'sample_{}'.format(idx)
    os.makedirs('./model_inputs', exist_ok=True)
    with open(os.path.join('./model_inputs', sample_name), 'wb') as f:
        pickle.dump(temp, f)
    info.append({"sample": "model_inputs/{}".format(sample_name)})

with open('./cali_data.json', 'w', encoding='utf-8') as f:
    json.dump(info, f, indent=2, ensure_ascii=False)
```

迁移适配新模型时，复制一个最接近的 demo 的 `make_calidata.py`，只需改三处：模型加载方式、`hook_module`（待量化子模块，如 `model.model` / `model.visual`）、`build_inputs`（构造模型输入），捕获调用直接复用 `capture_module_input` 即可。`run` 也可以直接传模型方法并透传参数：`capture_module_input(model.model, model.generate, **inputs, max_new_tokens=128)`。

注： 1. `capture_module_input` 的第一个参数需为**量化模型本身**（即调用 `load_model` 时传入的子模块，如纯 LLM 的 `model`、VLM 的 `model.model` / `model.model.text_model` / `model.visual` 等），捕获的是该子模块**最外层入口的输入**，而非其内部第一个 transformer block 的输入。RKQuantizer 量化时会将这份输入整体喂给该子模块，再在其内部逐层统计激活，因此捕获点必须与 `load_model` 传入的对象一致。 2. 浮点模型加载时的 `torch_dtype` 应与量化时保持一致，否则捕获的输入数值精度可能与量化阶段不匹配。 3. 内部 hook 抛出的 `StopForward` 会中断前向，函数已内部 `try/except` 处理，调用方无需再包裹；hook 在函数返回前一定被移除，不会泄漏到后续调用。 4. 若不使用公共工具而自行实现 hook，需在 pre hook 中 `raise` 一个异常以终止前向，并自行管理 hook 的注册与移除。 5. 推荐校准样本数为 20~128 条，样本应尽量覆盖实际部署场景的输入分布。

方式二：纯文本 JSON（仅适用于纯文本 LLM）

对于输入仅为文本的 LLM 模型，可采用一种更简单的方式：JSON 文件中每条样本为一个包含 `input` 与 `target` 两个键的字典，RKQuantizer 内部会自动使用 `tokenizer(input + target)` 将其编码为模型输入。该方式无需加载浮点模型、无需 hook，准备成本最低。格式如下：

```
[
  {
    "input": "半胱氨酸和酪氨酸在人体内可分别由什么转变而来。\nA. 胱氨酸和蛋氨酸\nB. 蛋氨酸和苯丙氨酸",
    "target": "\n答案:\n B"
  },
  {
    "input": "Natalia sold clips to 48 of her friends in April, and then she sold half as many in May. How many altogether?",
    "target": ""
  }
]
```

其中 `input` 为提示文本，`target` 为期望的输出文本（可为空字符串）；二者拼接后整体送入 tokenizer 编码，`target` 中可包含答案或思维链等补充上下文，使校准分布更贴近真实生成场景。举例如下：

```python
import json

# 直接手写或从已有数据集整理，保存为 JSON 即可
samples = [
    {"input": "请简述牛顿第一定律的内容。", "target": "\n答案:\n 牛顿第一定律指出..."},
    {"input": "1+1等于几? ", "target": "\n答案:\n 2"},
]
with open('./cali_data.json', 'w', encoding='utf-8') as f:
    json.dump(samples, f, ensure_ascii=False, indent=2)
```

注： 1. 该方式仅适用于纯文本输入的 LLM。对于多模态（VLM）、语音等含非文本输入的模型，必须使用方式一。

校准数据的选择

校准数据的质量极大影响量化后模型的精度，选择时应遵循核心原则：**使用真实应用场景下的多样化数据**，使校准数据的分布尽量贴近模型部署后的实际输入分布。具体建议如下：

1. **覆盖真实场景**：校准样本应来自模型实际部署场景的真实输入（实际的 system prompt、function-tool(**对agent任务十分重要**)等 可以通过 load_model 函数接口加载），而非任意的通用语料。
2. **保持多样性**：样本应具有多样性，避免内容高度同质化，以保证量化参数能够适应不同的输入分布。
3. **区分模型类型**：
4. 对于**开源通用 base 模型**，校准数据的影响相对较小，使用通用语料通常即可获得可接受的量化精度；
5. 对于**在垂类任务上微调过的模型**，校准数据的影响则非常大。举例来说，垂类微调后的模型对 system prompt 以及结构化数据（如 JSON、表格、特定格式指令等）特别关注，若在量化（GRQ 微调）阶段不喂入这类数据，会破坏模型在垂类任务上的能力，导致量化后垂类任务表现明显下降。因此垂类微调模型必须使用与实际垂类任务一致的 system prompt 和结构化数据作为校准样本。

11.3 AWQ 量化配置

AWQ（Activation-aware Weight Quantization）是一种基于激活分布对权重进行缩放校正的量化算法，可在低位量化（如 w4a16）下进一步降低精度损失。在 RKQuantizer 中，当 `quantize` 接口的 `quantized_algorithm='grq'` 且 `awq_config` 不为 None 时，会先执行 AWQ 再执行 GRQ；`awq_config` 为 None 时跳过 AWQ。

`awq_config` 是一个字典，用于描述模型 transformer block 的内部结构，使 AWQ 算法能够定位每个 block 中的注意力相关 Linear 层、FFN 相关 Linear 层以及前后 LayerNorm。其字段说明如下：

字段	说明
BLOCKNAME	block 模块名，用于识别模型的 transformer block（如 'Qwen2DecoderLayer'、'LlamaDecoderLayer'）。AWQ 以 block 为单位逐层校正，**该字段必填**。
ATTN_NORM	注意力模块前的 LayerNorm 子模块名（如 'input_layernorm'），用于定位 attention 输入来源。
ATTN_Q	注意力 Q 投影 Linear 子模块名（如 'self_attn.q_proj'）。当 Q/K/V 为独立投影时填写，合并投影时填空字符串。
ATTN_K	注意力 K 投影 Linear 子模块名（如 'self_attn.k_proj'）。同 ATTN_Q。
ATTN_V	注意力 V 投影 Linear 子模块名（如 'self_attn.v_proj'）。同 ATTN_Q。
ATTN_QKV	注意力 QKV 合并投影 Linear 子模块名（如 'attn.qkv'）。当 Q/K/V 合并为单个 Linear 时填写此项，并将 ATTN_Q/ATTN_K/ATTN_V 置空。
ATTN_KV	注意力 KV 合并投影 Linear 子模块名。当 K/V 合并（Q 独立）时填写，其余置空。
ATTN_OUT	注意力输出投影 Linear 子模块名（如 'self_attn.o_proj'、'attn.proj'）。
FFN_NORM	FFN 模块前的 LayerNorm 子模块名（如 'post_attention_layernorm'），用于定位 FFN 输入来源。
FFN_UP	FFN up 投影 Linear 子模块名（如 'mlp.up_proj'）。无 gate 结构的 FFN 填写，gate 结构时填空。
FFN_GATE	FFN gate 投影 Linear 子模块名（如 'mlp.gate_proj'）。带 gate 的 FFN（如 LLaMA 系 SwiGLU）填写，用于定位 FFN 输入来源。
FFN_DOWN	FFN down 投影 Linear 子模块名（如 'mlp.down_proj'、'mlp.linear_fc2'）。

注：
1. 注意力投影有三种常见结构，需按实际模型选择填写其一，其余置空：
- Q/K/V 独立投影：填 ATTN_Q/ATTN_K/ATTN_V，ATTN_QKV/ATTN_KV 置空；
- QKV 合并投影：填 ATTN_QKV，ATTN_Q/ATTN_K/ATTN_V/ATTN_KV 置空；
- KV 合并投影（Q 独立）：填 ATTN_Q 与 ATTN_KV，其余置空。
2. FFN 同样需按结构选择：带 gate 的 FFN（如 SwiGLU）填 FFN_GATE/FFN_DOWN，FFN_UP 置空；不带 gate 的 FFN 填 FFN_UP/FFN_DOWN，FFN_GATE 置空。
3. 各字段值为 block 内子模块的相对路径（相对 block 模块），与模型 `named_modules()` 中的命名一致，可通过打印模型结构获取。
4. `awq_config` 仅在 `quantized_algorithm='grq'` 且配置不为 None 时生效。
5. AWQ 与混合精度量化（`layer_quant_config` / `auto_hybrid_rate`）可同时使用，二者互不影响。
6. **并非任意模型都能使用 AWQ。** AWQ 通过对权重缩放校正来降低量化误差，其校正逻辑要求 transformer block 内的关键 Linear 层（attention 的 Q/K/V/QKV/KV/out、FFN 的 up/gate/down）的前置算子为 LayerNorm、Linear。当模型结构不满足该前提时（例如关键 Linear 前置了其它自定义算子，或 block 结构无法被 `BLOCKNAME` 正确识别），内部会抛出异常。因此在使用 AWQ 前应先确认模型结构可被

`awq_config` 正确描述；对于不支持 AWQ 的模型，可设 `awq_config=None` 仅使用 GRQ 量化。**目前 Qwen3.5, Gemma4模型 无法使用 AWQ。**

Q/K/V 独立投影的 LLM 模型配置示例（如 Qwen2.5、Llama 系）：

```
awq_config = {
    "BLOCKNAME": "Qwen2DecoderLayer",
    "ATTN_NORM": "input_layernorm",
    "ATTN_Q": "self_attn.q_proj",
    "ATTN_K": "self_attn.k_proj",
    "ATTN_V": "self_attn.v_proj",
    "ATTN_QKV": "",
    "ATTN_KV": "",
    "ATTN_OUT": "self_attn.o_proj",
    "FFN_NORM": "post_attention_layernorm",
    "FFN_UP": "mlp.up_proj",
    "FFN_GATE": "mlp.gate_proj",
    "FFN_DOWN": "mlp.down_proj"
}

model = QuantTool.quantize(quantized_dtype="w4a16",
                           quantized_method="group32",
                           quantized_algorithm='grq',
                           dataset='./dataset.json',
                           awq_config=awq_config)
```

QKV 合并投影的视觉模型配置示例（如 Qwen3-VL、Qwen2.5-VL 的 vision 部分）：

```
awq_config = {
    "BLOCKNAME": "Qwen3VLVisionBlock",
    "ATTN_NORM": "norm1",
    "ATTN_Q": "",
    "ATTN_K": "",
    "ATTN_V": "",
    "ATTN_QKV": "attn.qkv",
    "ATTN_KV": "",
    "ATTN_OUT": "attn.proj",
    "FFN_NORM": "norm2",
    "FFN_UP": "",
    "FFN_GATE": "mlp.linear_fc1",
    "FFN_DOWN": "mlp.linear_fc2"
}

model = QuantTool.quantize(quantized_dtype="w4a16",
                           quantized_method="group32",
                           quantized_algorithm='grq',
                           dataset='./dataset.json',
                           awq_config=awq_config)
```

关于 AWQ 的精度增益

根据实测，在 w4a16 + group32 的设置下，开启 AWQ（即 `awq_config` 不为 None）带来的精度增益通常不明显，部分情况下甚至可能出现轻微精度劣化。因此 AWQ 并非默认必开项，建议遵循以下原则：**优先仅使用 GRQ 量化，当 GRQ 量化后精度仍不满足要求时，再考虑开启 AWQ**，并通过对比实测精度决定是否保留 AWQ 结果。

11.4 混合精度量化

当模型整体采用较低位量化（如 w4a16）时，部分对量化敏感的层（如注意力中的 v_proj、FFN 中的 down_proj、特定结构的线性注意力层等）可能出现明显的精度损失。混合精度量化允许将这些敏感层保持较高精度，其余层仍按 `quantized_dtype` 指定的类型量化，从而在精度与模型体积/精度之间取得平衡。RKQuantizer 提供两种混合精度量化方式：手动按层指定（`layer_quant_config`）与自动按比例挑选（`auto_hybrid_rate`）。

方式一：手动按层指定（layer_quant_config）

由用户显式指定哪些层使用 w6a16 或者 w8a16。`layer_quant_config` 可传入字典或 JSON 文件路径，格式为 `{layer_pattern: "w8a16"}`，其中 `layer_pattern` 为用于匹配 Linear 层名称的 shell-style 通配模式，支持 `*` 通配符（用于跨所有 transformer block 匹配同名子层）。匹配到的层将使用配置中指定的量化类型，未匹配到的层按 `quantized_dtype` 量化。当前 `layer_quant_config` 仅支持 "w8a16" 或者 "w6a16"，且不可与 `auto_hybrid_rate` 同时使用。

以 Qwen3.5 模型为例，其 `layer_quant_config.json` 内容如下，将线性注意力相关的 qkv/b/a/out_proj 层以及 v_proj、down_proj 层指定为 w8a16 (或者 w6a16)：

```
{
  "model.layers.*.linear_attn.in_proj_qkv": "w8a16",
  "model.layers.*.linear_attn.in_proj_b": "w8a16",
  "model.layers.*.linear_attn.in_proj_a": "w8a16",
  "model.layers.*.linear_attn.out_proj": "w8a16",
  "model.layers.*.self_attn.v_proj": "w8a16",
  "model.layers.*.mlp.down_proj": "w8a16"
}
```

对应量化调用如下：

```
quant_model = QuantTool.quantize(quantized_dtype="w4a16",
                                 quantized_method="group32",
                                 quantized_algorithm='grq',
                                 dataset='./dataset.json',
                                 awq_config=None,
                                 layer_quant_config="layer_quant_config.json",
                                 auto_hybrid_rate=0.)
```

方式二：自动按比例挑选（auto_hybrid_rate）

无需逐层指定，由 RKQuantizer 根据校准数据自动评估各层对量化的敏感度，并按指定比例将最敏感的若干 Linear 层保持 w8a16。`auto_hybrid_rate` 取值为 [0, 1) 的浮点数，表示被设为 w8a16 的 Linear 层占比，例如 0.2 表示约 20% 的 Linear 层会被设为 w8a16，其余按 `quantized_dtype` 量化。该参数不可与 `layer_quant_config` 同时使用。举例如下：

```
## auto_hybrid_rate=0.2 表示 20% 的 Linear 层会被设置为 w8a16 类型
model = QuantTool.quantize(quantized_dtype="w4a16",
                           quantized_method="group32",
                           quantized_algorithm="grq",
                           dataset='./dataset.json',
                           auto_hybrid_rate=0.2,
                           awq_config=None)
```

导出各层量化类型（export_op_quantized_dtype）

无论采用上述哪种混合精度方式，量化完成后都需要调用 `export_op_quantized_dtype` 将各层实际使用的量化类型导出为 JSON 文件，供后续 `rknn.build` 阶段对齐使用。该接口需要传入导出后的 ONNX 模型路径，用于建立 torch 层名与 ONNX 算子之间的映射；导出的 JSON 为 `{onnx_op_name: dtype}` 格式，key 为 ONNX 模型中 MatMul/Gemm 等算子的名称，value 为该算子的量化类型。以 Qwen3.5 为例，导出的 `layer_bit.json` 内容如下：

```
{
  "onnx::MatMul_8968": "w8a16",
  "onnx::MatMul_8973": "w8a16",
  "onnx::MatMul_9657": "w8a16",
  ...
}
```

举例如下：

```
# 在导出 ONNX 之后调用，传入 ONNX 模型路径
QuantTool.export_op_quantized_dtype(onnx_path='./Qwen3.5-0.8B-llm.onnx',
                                    op_dtype_path='layer_bit.json')
```

导出 RKNN 模型时的对应修改

由于混合精度量化已在 RKQuantizer 阶段完成（ONNX 权重已按各层类型完成 fake-quant），在通过 `rknn.config` + `rknn.load_llm` + `rknn.build` 导出 RKNN 模型时，需要通过 `rknn.config` 的 `op_quantized_dtype` 参数将 `layer_bit.json` 传入，使 RKNN 转换阶段按各层指定的量化类型还原，避免转换阶段对敏感层重新做低位量化导致精度回退。`op_quantized_dtype` 接受字典或 JSON 文件路径，目前仅对 MatMul/Gemm/Linear 算子生效，支持的取值为 w6a16 或者 w8a16。

```
rknn.config(
    target_platform='rk1820',
    quantized_dtype='w4a16',
    quantized_algorithm='normal',
    quantized_method='group32',
    llm_config=llm_config,
    op_quantized_dtype='./layer_bit.json',    # 传入混合精度各层量化类型
)
```

注： 1. `layer_quant_config` 与 `auto_hybrid_rate` 不可同时使用。 2. `export_op_quantized_dtype` 必须在 ONNX 导出完成之后调用，且传入的 `onnx_path` 应与后续 `rknn.load_llm` 加载的 ONNX 一致，否则 torch 层名与 ONNX 算子的映射会失败。 3. 导出 RKNN 时 `op_quantized_dtype` 指定的 JSON 必须与量化阶段 `export_op_quantized_dtype` 导出的文件对应，二者缺一会导致混合精度配置不生效或转换报错。 4. 未开启混合精度量化时，无需调用 `export_op_quantized_dtype`，也无需在 `rknn.config` 中配置 `op_quantized_dtype`。

11.5 GRQ 量化显存占用说明

GRQ 量化算法会对模型权重进行微调，其执行过程涉及单个 transformer block 的前向推理与反向传播，因此显存占用主要来源于单个 block 的前向/反向计算，而非整个模型同时驻留。

影响显存的关键因素

- **校准样本数量**：GRQ 逐 block、逐样本进行微调，峰值显存由单个 block 单次计算决定，与样本数量基本无关。因此增加样本数不会显著抬高峰值显存，样本数也无需过大，**一般无需超过 200 个**。
- **校准样本 token 数**：样本的 token 长度与显存占用呈正相关。由于 Transformer 的注意力计算复杂度为 $O(T^2)$，token 数越大，单次前向/反向所需的中间激活与显存增长越快，是决定峰值显存的主要因素。因此准备校准数据时需结合当前显卡的实际显存容量，合理控制单条样本的 token 长度。

降低显存占用的方法

加载模型时指定 FlashAttention2 可有效减少 GRQ 阶段的显存占用（FlashAttention 将注意力的 $O(T^2)$ 显存开销降至 $O(T)$），建议在显存受限时启用。启用方式为在 `from_pretrained` 时设置 `_attn_implementation="flash_attention_2"`，例如：

```
model = AutoModelForCausalLM.from_pretrained(
    args.model_path,
    torch_dtype=torch.bfloat16,  ## 开启 FlashAttention2 要求模型加载类型必须为bfloat16
    _attn_implementation="flash_attention_2",   # 开启 FlashAttention2 以降低 GRQ 显存
占用
    trust_remote_code=True
)
```

注意：FlashAttention2 仅用于 GRQ 量化阶段以降低显存。在导出 ONNX 之前，需将 attention 实现方式重置为 eager 模式（ONNX 导出不支持 FlashAttention），例如：

```
# 导出 ONNX 前将 attention 切回 eager
model.config._attn_implementation = "eager"
for layer in model.model.layers:
    layer.self_attn.config._attn_implementation = "eager"
```

实测显存参考

以下为不同 attention 实现下的实测显存占用（供参考）。未开启 FlashAttention2（默认 eager attention）时，注意力计算的 $O(T^2)$ 显存开销无法降至 $O(T)$，显存占用会显著高于开启时，显存受限的显卡建议启用 FlashAttention2：

模型	校准样本 token 长度	开启 FlashAttention2	不使用 FlashAttention2（默认 eager）
Qwen3-4B	约 13K	约 7G	约 20G
Qwen3.5-4B	约 13K	约 15G	约 30G